Project Id : 10

A

Project Report On

Project Name

Submitted in the Partial fulfillment of the requirement for awarding the degree of Bachelor of Computer Application (BCA)
SEM 5, Year 2026-27

By

Bhautik Vachhani

Under the Guidance of

Irfan Belim

Faculty of Computer Application,
Noble BCA College
Noble UNIVERSITY – JUNAGADH



Noble University, Junagadh

Certificate

Enrollment No. 240431247

This is to certify that the Project / practical work was satisfactorily carried out and hence submitted this report is the bonafide work of Mr./ Ms. Bhautik Vachhani

Student of Bachelor of Computer Applications Sem 5 in the Laboratory of Noble University, Junagadh during the academic year 2026-27.

Internal Guide — Irfan belim Sir

Principal — Bhargav Rajyagor

Preface

This report contains all the outputs of a project undertaken to develop a working model of **E- Commerce Website** and to study the existing working system. As part of this study, we analyzed the requirements of the working model and tried to improve it using the technological functionality that the current computer science provides.

This volume contains the documentation summary of the working system, the background study and analysis, the technical requirements and the working specification of the developed system.

We recommend the user to take the advantage of the study presented here and implement the system with their required improvisation. We hope this study will reduce the burden of the reanalysis of the development phase.

Acknowledgment

"It is not possible to prepare a project report without the assistance & encouragement of other people. This one is certainly no exception."

On the very outset of this report, We would like to extend our sincere & heartfelt obligation towards all the personages who have helped us in this endeavor. Without their active guidance, help, cooperation & encouragement, We would not have made headway in the project. We are ineffably indebted to our Dean **Mr. Paresh Vora** Sir for his conscientious guidance and encouragement to accomplish this assignment.

We are extremely thankful and pay our gratitude to our project guide Prof **Irfan belim Sir** his/her valuable guidance and support on completion of this project in its presently.

We extend our gratitude to Faculty of Computer Application, Noble University, Junagadh for giving us this opportunity.

Thanking You

Student-1 Bhautik Vacahhani (240431247)

INDEX

Sr. No.	Title		Page No.
1	**Project Profile**		
	1.1	Definition	
	1.2	Scope	
	1.3	Objective	
2	**System Analysis and Specification**		
	2.1	Existing System	
	2.2	Limitations of Existing System	
	2.3	Feasibility Study	
	2.4	Need of New System	
	2.5	Front End & Back End Tools	
3	**System Requirement Specification**		
	3.1	Proposed System and Advantages	
	3.2	Development Strategy Model	
4	**System Design**		
	4.1	Data Flow Diagram	
	4.2	E-R Diagram	
	4.3	Class Diagram	
	4.4	Use Case Diagram	
	4.5	Sequence Diagram	
	4.6	Activity Diagram	
	4.7	Data Dictionary (Database)	
5	**Screen Design**		
	5.1	Input Design	
	5.2	Output Design/ Report Design	
6	**Coding (Few Code Snippets)**		
7	**Testing (Manual, Test Cases and Test Data)**		

	7.1	Manual Testing	
	7.2	Test Cases	
8	**Enhancements**		
	8.1	Advantages of Your Project	
	8.2	Limitation of Your Project	
	8.3	Future Scope	
9	**References**		

1. Project Profile

Project on : E-commerce Website

Developed By : Bhautik Vachhani

Project id : 10

Enrollment No : 240431247

Branch : BCA (sem-5)

Guider By : prof. Irfan belim sir

Technology :

- **Frontend:** Recat.js
- **Backend:** Node.js, Express.js
- **Database:** MongoDB

Submitted To : Faculty of Computer Application, Noble University

1.1 Definition

Style Hub is a full-stack online fashion e-commerce web application designed to provide users with a seamless and modern shopping experience for fashion and clothing products. The platform enables customers to browse, search, and purchase fashion items including men's and women's clothing, accessories, and more through a responsive and intuitive web interface.

Style Hub acts as a digital marketplace connecting fashion retailers (administrators) with consumers. The platform facilitates the complete e-commerce lifecycle — from product discovery and selection to cart management, checkout, and order tracking. The application features a dual-interface system: a customer-facing storefront for end-users and an administrative dashboard for managing products, orders, and business operations.

Built using the MERN Stack (MongoDB, Express.js, React.js, Node.js), Style Hub leverages modern web technologies to deliver a fast, scalable, and secure e-commerce experience. The system incorporates features such as Google OAuth authentication, Wishlist management, and real-time order tracking — making it a comprehensive solution for online fashion retail.

The name "Style Hub" reflects the platform's core mission: to serve as a centralized hub where fashion-conscious users can discover and purchase stylish clothing and accessories with ease. The tagline "Fashion That Defines You" encapsulates the brand's commitment to helping users express their personal style through curated fashion offerings.

1.2 Scope

The scope of the Style Hub project encompasses the following key areas:

1. **User Management:**

    - User registration with email/password (local authentication)
    - Google OAuth 2.0 social login integration
    - Secure login/logout functionality
    - Password recovery via email (Forgot Password with Node mailer)
    - Profile management with profile picture support
    - Address management for delivery

2. **Product Management (Admin):**

- Add new products with multiple images (up to 5 images per product)
- Edit/update existing products
- Delete products from catalog
- Manage product discounts
- View all products in admin panel

3. **Shopping Experience (Customer):**

- Browse all products on the shop page
- Category-wise browsing (Men's collection, Women's collection)
- Product detail view with image gallery
- Search and filter products
- Discount/sale product filtering

4. **Cart & Wishlist:**

- Add/remove products to/from cart
- Increase/decrease cart item quantity
- Add/remove products to/from Wishlist
- Persistent cart and Wishlist (stored in database)

5. **Order Management:**

- Multi-step checkout process
- Address selection for delivery
- GST calculation (SGST, CGST for Gujarat; IGST for other states)
- Shipping charge calculation
- Cash on Delivery (COD) payment option
- Order placement and confirmation
- Order history and tracking
- Order cancellation
- Admin order status management (Placed → Processing → Shipped → Delivered)

6. **Informational Pages:**

- FAQ page
- Privacy Policy page
- Terms and Conditions page
- Customer Support/Contact page

1.3 Objective:

The primary objectives of the Style Hub project are:

- **To develop a modern, user-friendly e-commerce platform:**
  that enables customers to browse, select, and purchase fashion products online with an intuitive and visually appealing interface.

- **To implement a secure authentication system:**
  that supports multiple login methods including traditional email/password registration and Google OAuth 2.0 social login, ensuring user data privacy and account security.

- **To create a robust product management system:**
  that allows administrators to efficiently add, edit, delete, and manage fashion products with multiple images, descriptions, pricing, categories, and discounts.

- **To build a seamless shopping cart and checkout experience:**
  with features like quantity management, Wishlist functionality, address management, and automatic tax (GST) and shipping calculations based on the delivery location.

- **To integrate a reliable payment gateway:**
  that supports online payments alongside Cash on Delivery options, providing customers with flexible payment choices.

- **To implement a comprehensive order management system:**
  that enables customers to track their orders and allows administrators to update order statuses through the complete fulfilment lifecycle (Placed → Processing → Shipped → Delivered → Cancelled).

- **To ensure application security:**
  by implementing industry-standard practices including password hashing (bcrypt), JWT-based authentication with HTTP-only cookies, Helmet.js security headers, and input validation.

- **To develop a responsive web application:**
  using modern frontend technologies (React.js, Tailwind CSS) that provides an optimal viewing experience across desktop and mobile devices.

- **To implement an email notification system:**
  using Node mailer for password reset functionality, enhancing the user experience with account recovery capabilities.

- **To build a scalable architecture:**
  using the MERN stack that can accommodate future feature additions and increasing user traffic without significant refactoring.

2. System Analysis and Specification

2.1 Existing System

In the current market, several e-commerce platforms exist for fashion and clothing retail. Some of the prominent existing systems include:

1. **Amazon Fashion:**
   A global marketplace offering a vast selection of clothing and accessories from multiple brands. However, it is a general-purpose platform not exclusively focused on fashion, leading to a cluttered browsing experience for fashion-specific shopping.

2. **Flipkart Fashion / Myntra:**
   Popular in India for fashion e-commerce. These platforms are large-scale, enterprise-level applications with complex architectures not suitable for small-to-medium fashion retailers who want their own branded store.

3. **Ajio / Nykaa Fashion:**
   Niche fashion platforms that cater to specific demographics. They are proprietary systems that cannot be customized or self-hosted by independent fashion businesses.

4. **Shopify-based Stores:**
   Many small fashion businesses use Shopify to create online stores. However, Shopify charges monthly subscription fees and transaction fees, which may not be cost-effective for emerging fashion businesses.

Traditional Retail System:
Before e-commerce, fashion retail relied entirely on physical brick-and-mortar stores. Customers had to physically visit stores, browse through limited inventory, and purchase items in person. This system had inherent limitations:

- **Limited reach**: Only customers within geographic proximity could shop.
- **Fixed operating hours**: Customers could only shop during store hours.
- **Limited inventory display**: Physical space constraints limited the number of products that could be displayed.
- **No product comparison**: Customers could not easily compare products, prices, or reviews.
- **Manual record-keeping**: Inventory, orders, and customer data were managed manually, prone to errors.

2.2 Limitations of Existing System

The existing systems and traditional approaches have several significant limitations:

- **Limitations of Traditional Retail:**

1. **Geographical Limitations:**
   Physical stores are limited to serving customers within a specific geographical area, missing out on potential customers from other regions.

2. **Time Constraints:**
   Brick-and-mortar stores operate within fixed business hours, making it inconvenient for customers who prefer shopping outside these hours.

3. **Limited Product Display:**
   Physical stores can only display a limited number of products due to space constraints. Customers cannot browse the complete catalog.

4. **Manual Inventory Management:**
   Traditional systems rely on manual record-keeping for inventory, sales, and customer information, leading to errors and inefficiencies.

5. **No Personalized Experience:**
   Traditional stores cannot provide personalized recommendations, Wishlist, or saved shopping carts for individual customers.

6. **High Operational Costs:**
   Rent, utilities, staffing, and maintenance of physical stores result in high operational overhead that is passed on to customers through higher product prices.

- **Limitations of Existing E-Commerce Platforms:**

1. **High Platform Fees:**
   Platforms like Shopify and Amazon charge monthly subscriptions, listing fees, and transaction commissions (typically 5-15%), significantly reducing profit margins for small fashion businesses.

2. **Limited Customization:**
   Third-party platforms restrict design and functionality customization, preventing businesses from creating a unique brand experience.

3. **Data Dependency:**
   Seller data, customer information, and analytics are controlled by the platform, creating dependency and limiting business insights.

4. **Complex User Interfaces:**
   Large platforms like Amazon have cluttered interfaces with thousands of categories, making it overwhelming for users looking specifically for fashion products.

5. **No Direct Customer Relationship:**
   Marketplaces like Amazon and Flipkart act as intermediaries, preventing direct communication and relationship building between the fashion brand and its customers.

6. **Lack of Indian Tax Compliance:**
   Many international platforms do not natively support Indian GST calculations (SGST, CGST, IGST) and state-based tax rules.

7. **Cookie-Cutter Design:**
   Template-based stores (Shopify) often look similar, failing to create a distinctive brand identity for the fashion business.

8. **Vendor Lock-in:**
   Once a business sets up on a platform, migrating to another system is complex and time-consuming, creating dependency.

2.3 Feasibility Study

A feasibility study is conducted to determine whether the proposed system is viable from technical, economic, and operational perspectives. The three types of feasibility studies conducted for Style Hub are:

2.3.1 Technical Feasibility:

Technical feasibility evaluates whether the required technology, tools, and expertise are available to develop the system.

Aspect	Assessment
Frontend Technology	React.js 19 is a widely adopted, open-source JavaScript library with extensive community support and documentation. Vite provides fast build tooling.
Backend Technology	Node.js with Express.js 5 is a mature, well-documented framework for building RESTful APIs.
Database	MongoDB is a free, open-source NoSQL database hosted on MongoDB Atlas (cloud).
Authentication	JWT and Google OAuth 2.0 are industry-standard authentication protocols with well-maintained libraries (jsonwebtoken, google-auth-library).
Email Service	Node mailer with Gmail SMTP provides free email sending capability.
File Upload	Multer is a mature Node.js middleware for handling multipart/form-data for file uploads.
Styling	Tailwind CSS 4 is an open-source utility-first CSS framework.
Development Tools	VS Code (IDE), Git (version control), npm (package management) are freely available.

Hosting	The application can be deployed on free/affordable platforms (Render for frontend and backend, MongoDB Atlas for database).

2.3.2 Economic Feasibility:

Economic feasibility evaluates whether the project can be developed and maintained within an acceptable budget.

Cost Category	Estimated Cost
Development Tools	₹0 (Open-source: VS Code, Node.js, React, MongoDB Community)
Domain Name	₹0 (Render Provide free Domain)
Hosting (Frontend)	₹0 (Render free tier)
Hosting (Backend)	₹0 (Render free tier)
Database (MongoDB Atlas)	₹0 (Free tier: 512 MB storage)
Email (Gmail SMTP)	₹0 (up to 500 emails/day)
SSL Certificate	₹0 (Included with hosting)
Total Initial Cost	**₹0 (Free)**

Benefits:

- Eliminates physical store rent and utility costs
- Reduces staff requirements for order processing
- Enables 24/7 business operation without additional labor costs
- Provides access to customers nationwide
- Automates inventory, order management, and billing

2.3.3 Operational Feasibility:

Operational feasibility evaluates whether the system can be effectively used and maintained in the real-world environment.

Aspect	Assessment
User Interface	Intuitive, modern UI built with React.js and Tailwind CSS. Minimal training required for customers.
Admin Panel	Dedicated admin dashboard with clear navigation for product, order, and user management.
Accessibility	Web-based application accessible via any modern browser (Chrome, Firefox, Safari, Edge) on desktop and mobile.
Scalability	MERN stack architecture supports horizontal and vertical scaling as business grows.
Maintenance	Modular codebase with clear separation of concerns (MVC pattern) allows easy maintenance and updates.
Security	Industry-standard security practices (JWT, bcrypt, Helmet.js, CORS) ensure data protection.
Backup	MongoDB Atlas provides automated backups and data redundancy.
User Adoption	Google OAuth login reduces friction; familiar e-commerce patterns increase user comfort.

2.4 Need of New System

Despite the availability of existing e-commerce platforms, there is a clear need for a custom-built system like StyleHub for the following reasons:

1. **Brand Identity and Customization:**
   Existing marketplace platforms (Amazon, Flipkart) do not allow sellers to create a unique brand experience. StyleHub provides a dedicated, branded storefront that reflects the fashion business's unique identity, aesthetic, and values. The tagline "Fashion That Defines You" is integrated throughout the customer experience.

2. **Cost-Effective Solution:**
   Third-party platforms charge significant fees (monthly subscriptions, transaction commissions, listing fees) that eat into profit margins. StyleHub eliminates these recurring platform fees — the only costs are minimal hosting and payment gateway transaction charges (2%).

3. **Full Data Ownership:**
   With marketplace platforms, customer data and analytics belong to the platform. StyleHub gives the business complete ownership of customer data, order history, and behavioral analytics, enabling better marketing and business decisions.

4. **Indian Market Compliance:**
   StyleHub is specifically designed for the Indian market with built-in GST calculation logic:

   - **Intra-state (Gujarat):** SGST (9%) + CGST (9%) = 18% GST, free shipping
   - **Inter-state (other states):** IGST (18%), ₹100 shipping charge

5. **Dual Authentication System:**
   StyleHub supports both traditional email/password registration and Google OAuth 2.0 social login. This hybrid approach reduces registration friction (Google login) while maintaining a traditional option for users who prefer it.

6. **Admin Control Panel:**
   A dedicated admin dashboard allows the business owner to:

   - Add, edit, and delete products with multiple images
   - Manage product discounts and pricing
   - Track and update order statuses
   - View all customer orders

7. **Modern Technology Stack:**
   StyleHub leverages the latest versions of industry-leading technologies:

   - React.js 19 for a fast, reactive user interface

- Express.js 5 for robust API development
- MongoDB for flexible, schema-less data storage
- Tailwind CSS 4 for responsive, modern styling

8. Scalability for Growth:

Unlike template-based solutions that hit limitations as the business grows, Style Hub's MERN stack architecture can scale horizontally to handle increasing traffic and product catalos.

9. Enhanced User Experience:

StyleHub provides features specifically designed for fashion e-commerce:

- Wishlist for saving favourite items
- Multiple product images per product (up to 5)
- Category-based browsing (Men, Women)
- Detailed product pages with description and pricing
- Order tracking from placement to delivery

2.5 Front End & Back End Tools

- **Front End Tools:**

Tool/Technology	Version	Purpose
React.js	19.2.7	Core frontend library for building component-based user interfaces with virtual DOM for optimal performance
Vite	8.1.1	Next-generation frontend build tool providing fast development server with Hot Module Replacement (HMR) and optimized production builds
Tailwind CSS	4.3.2	Utility-first CSS framework for rapid, responsive UI development without writing custom CSS
React Router DOM	7.18.1	Client-side routing library for single-page application (SPA) navigation between pages
Axios	1.18.1	Promise-based HTTP client for making API requests to the backend server
@react-oauth/google	0.13.5	Google OAuth 2.0 provider component for implementing Google social login
Font Awesome	7.3.0	Icon library providing scalable vector icons for UI elements (cart, Wishlist, user, etc.)

ESLint	10.6.0	Code linting tool for identifying and fixing JavaScript/React code quality issues

- **Back End Tools:**

Tool/Technology	Version	Purpose
Node.js	Latest LTS	JavaScript runtime environment for executing server-side code
Express.js	5.1.1	Minimal, flexible web application framework for building RESTful APIs
MongoDB	Cloud (Atlas)	NoSQL document database for flexible, scalable data storage
Mongoose	9.7.4	MongoDB object modelling (ODM) library for Node.js, providing schema-based data validation
JSON Web Token (jsonwebtoken)	9.0.3	Library for generating and verifying JWT tokens for secure authentication
bcryptjs	3.0.3	Library for hashing passwords using the bcrypt algorithm
google-auth-library	10.9.0	Google's official library for verifying Google OAuth tokens on the server

Multer	2.2.0	Middleware for handling `multipart/form-data` for file uploads (product images)
Nodemailer	9.0.3	Module for sending emails (password reset emails) via SMTP
Helmet	8.0.2	Security middleware that sets various HTTP headers to secure Express apps
CORS	2.8.6	Middleware for enabling Cross-Origin Resource Sharing between frontend and backend
cookie-parser	1.4.7	Middleware for parsing cookies attached to client requests
dotenv	17.4.2	Module for loading environment variables from a `. env` file into `process.env`

- **Development & Deployment Tools:**

Tool	Purpose
Visual Studio Code	Primary code editor/IDE for development
Git	Version control system for source code management
GitHub	Remote repository hosting for collaboration and backup
npm	Node Package Manager for installing and managing project dependencies
Postman	API testing tool for testing backend endpoints during development
MongoDB Compass	GUI tool for MongoDB database visualization and management
Chrome DevTools	Browser developer tools for frontend debugging and network analysis

3. SYSTEM REQUIREMENT SPECIFICATION

3.1 Proposed System and Advantages

- **Proposed System Overview:**

  The proposed system StyleHub is a comprehensive, full-stack e-commerce web application built using the MERN (MongoDB, Express.js, React.js, Node.js) technology stack. It is designed specifically for online fashion retail, providing a complete solution for both customers and administrators.

- **System Architecture:**

  StyleHub follows a three-tier architecture:

  1. **Presentation Tier (Frontend):**
     Built with React.js 19 and styled with Tailwind CSS 4. The frontend communicates with the backend via RESTful API calls using Axios. It handles user interface rendering, client-side routing (React Router DOM 7)

  2. **Application/Logic Tier (Backend):**
     Built with Node.js and Express.js 5. The backend handles business logic, authentication, authorization, file uploads, payment processing, and email notifications. It exposes RESTful API endpoints for the frontend to consume.

  3. **Data Tier (Database):**
     MongoDB (hosted on MongoDB Atlas) stores all application data including users, products, and orders. Mongoose ODM provides schema validation and data modelling.

- **Key Features of the Proposed System:**

A. Customer-Facing Features

Feature	Description
User Registration & Login	Dual authentication: traditional email/password and Google OAuth 2.0 social login
Forgot Password	Email-based password reset with secure, time-limited JWT tokens (5-minute expiry) and beautifully designed HTML email templates
Product Browsing	Browse all products, filter by category (Men/Women), view discounted products
Product Details	Detailed product page with multiple images, description, pricing, and discount information
Shopping Cart	Add/remove products, increase/decrease quantities, real-time price calculation
Wishlist	Save favourite products for future purchase; toggle add/remove functionality
Checkout	Multi-step checkout with address selection, GST calculation, and payment method selection
Payment	Cash on Delivery (COD) option
Order Tracking	View order history, track order status (Placed → Processing → Shipped → Delivered)
Order Cancellation	Cancel placed orders before they are shipped

Profile Management	View and update profile information, manage saved addresses
Informational Pages	FAQ, Privacy Policy, Terms & Conditions, Support/Contact page

B. Admin-Facing Features

Feature	Description
Admin Dashboard	Centralized dashboard for business management
Add Products	Add new products with name, description, price, category, discount, and up to 5 images
Edit Products	Update existing product details and images
Delete Products	Remove products from the catalog
Manage Discounts	Set and update discount percentages on individual products
View All Products	List view of all products with management options
Order Management	View all customer orders, update order statuses
Admin Navigation	Dedicated admin navbar with links to all management pages

- **Advantages of the Proposed System:**

1. Zero Platform Fees:
Unlike Shopify (₹2,000+/month) or Amazon (15% commission), StyleHub has no recurring platform costs. Only hosting (free tier available) and payment gateway (2% per transaction) costs apply.

2. Full Customization Control:
Being a custom-built application, every aspect — from UI design to business logic — can be modified to match exact business requirements.

3. Complete Data Ownership:
All customer data, order history, product information, and analytics are stored in the business's own MongoDB database, ensuring full data ownership and privacy compliance.

4. Dual Authentication:
Google OAuth integration reduces registration friction (one-click login), while traditional email/password provides a fallback option. Users who register via Google can also set a local password later.

5. Indian GST Compliance:
Built-in GST calculation logic handles both intra-state (SGST+CGST) and inter-state (IGST) tax computation, eliminating the need for third-party tax plugins.

6. Secure Architecture:
Multiple layers of security including:

- Password hashing with bcrypt (10 salt rounds)
- JWT authentication with HTTP-only, secure cookies
- Helmet.js for HTTP security headers
- CORS configuration to prevent unauthorized cross-origin requests
- Input validation on both client and server side

7. Responsive Design:
Tailwind CSS ensures the application is fully responsive across all screen sizes — desktop, tablet, and mobile — without separate codebases.

8. Fast Development & Build:
Vite's HMR provides near-instant development feedback, and optimized production builds ensure fast page load times for end-users.

9. Scalable NoSQL Database:
MongoDB's document-based storage provides flexible schema design, horizontal scalability, and excellent read/write performance for e-commerce workloads.

10. **Rich Email Templates:**
    Password reset emails use professionally designed HTML templates with the StyleHub branding, providing a polished user experience even in email interactions.

11. **Protected Routes:**
    Admin pages (Dashboard, Add Product, Show Products, Show Orders, Edit Product) are protected with role-based access control (RBAC), ensuring only authorized administrators can access management features.

12. **Multiple Product Images:**
    Each product supports up to 5 images (JPEG, JPG, PNG, WebP), allowing comprehensive product photography from multiple angles.

3.2 Development Strategy Model:

For the development of the StyleHub E-Commerce Website, the Agile Development Model (Iterative and Incremental Approach) was selected as the core software development strategy. Agile methodology allows the complete project to be divided into smaller and manageable development cycles called sprints. Each sprint focuses on implementing, testing, and improving a particular set of features.

The StyleHub project includes multiple modules such as user authentication, product management, shopping cart, wishlist, product filtering, checkout, address management, GST calculation, order management, Google OAuth login, password recovery, profile management, and admin functionality. Due to the number of modules and the possibility of changes during development, an iterative approach was suitable for the project.

The Agile model helped in developing the StyleHub application step-by-step. Each module was implemented and tested before moving to the next stage. This approach also made it easier to identify errors, improve existing features, and integrate additional functionality without affecting the complete application.

Key Phases of the Development Strategy Model:

1. **Requirement Analysis and Planning:**

   In the initial phase, the requirements of the StyleHub e-commerce website were identified and analyzed. The main objective was to develop an online shopping platform where customers can register, log in, browse products, add products to the cart or wishlist, complete checkout, make payments, and track their orders.

   The requirements of both Customer and Admin users were considered. Customer-side requirements included product browsing, category filtering, cart management, wishlist, checkout, payment, profile, address management, and order history. Admin-side requirements included product management and order management.

   The required technologies and project structure were also finalized during this phase.

2. **System Design and Architecture:**

   After requirement analysis, the overall structure and architecture of the StyleHub website were designed. The application was divided into frontend, backend, database, and authentication components.

   The frontend was designed using React and Vite, providing a responsive and interactive user interface. The backend was developed using Node.js and Express.js to handle API requests, business logic, authentication, product operations, cart operations, and order processing.

MongoDB was used as the database for storing users, products, cart information, wishlist information, addresses, and orders. Proper API routes and database models were planned to maintain communication between the frontend and backend.

3. Implementation and Coding (Iterative Development):

The implementation phase was divided into smaller modules and developed incrementally.

Module 1: User Authentication and Authorization:

The first module focused on user registration, login, logout, and authentication. JWT-based authentication was implemented using HTTP-only cookies. Role-based authorization was also implemented to differentiate between normal users and administrators.

Module 2: Product Management:

The product module was developed to manage product information such as product name, description, price, category, discount, image, and stock. Admin users can add, update, view, and delete products through the admin functionality. Product images are handled using image upload functionality.

Module 3: Product Browsing and Filtering:

The shopping cart module allows users to add products, remove products, and manage product quantities. Cart information is maintained according to the logged-in user.

The wishlist module allows users to save products that they may want to purchase later. Users can add or remove products from their wishlist.

Module 5: Checkout and Address Management:

The checkout module was developed to provide a complete order placement process. Users can select or manage their delivery address before placing an order. The system calculates the product subtotal, applicable GST components such as SGST, CGST, or IGST, shipping charges, and the final payable amount.

Module 6: Advanced Authentication and User Profile:

Additional authentication functionality was implemented using Google OAuth 2.0, allowing users to log in through their Google account.

Forgot-password and reset-password functionality was implemented using Nodemailer for sending password-related emails. A profile section was also developed where users can manage their personal information and saved addresses.

4. Testing and Quality Assurance:

After implementation of each module, testing was performed to verify its functionality. Individual features were tested before integrating them with other modules.

User registration and login were tested with valid and invalid credentials. Product operations were tested from the admin side. Cart operations such as adding products, changing quantities, and removing products were checked.

The checkout process was tested by verifying address details, GST calculation, shipping charges, payment processing, and order creation. Protected routes and role-based access were also tested to ensure that unauthorized users could not access admin functionality.

Responsive testing was performed to verify that the website works properly on different screen sizes such as desktop, tablet, and mobile devices.

5. Integration and Deployment:

After completing individual modules, all components were integrated into the complete StyleHub application. Frontend components were connected with backend APIs, and backend APIs were connected with the MongoDB database.

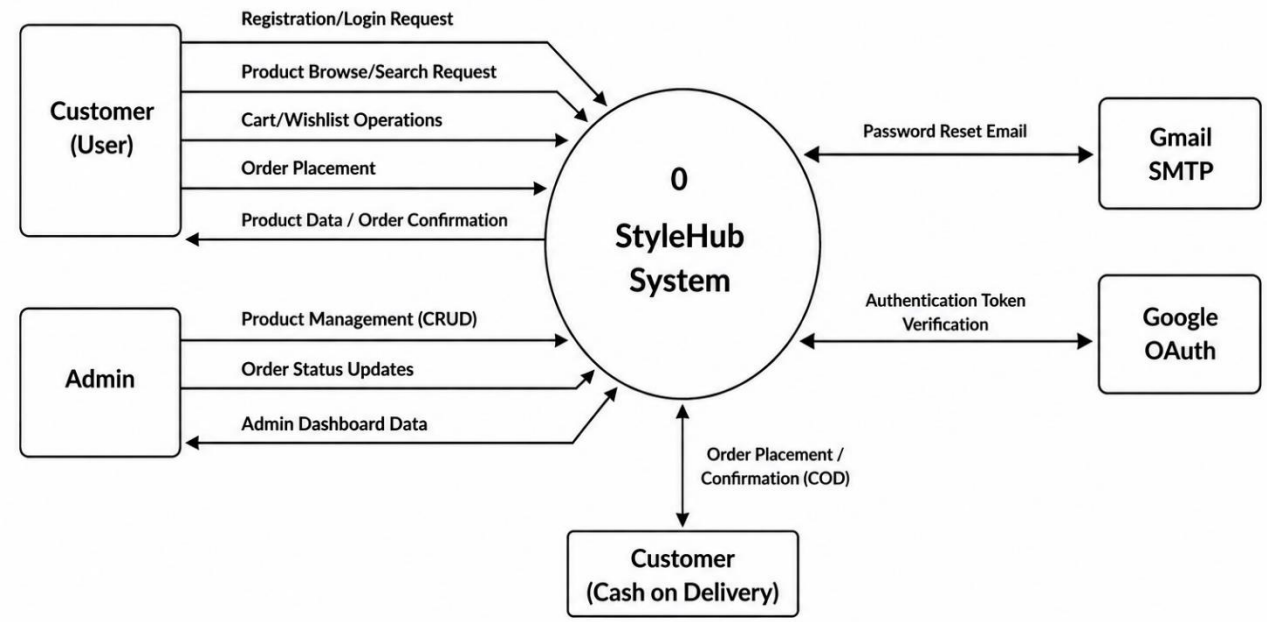
Authentication, product management, cart, wishlist, checkout, payment, and order modules were integrated and tested together to ensure smooth communication between different parts of the system.

After final testing and bug fixing, the application was prepared for deployment. Project documentation was also completed along with the final system.

4. System Design

4.1 Data Flow Diagram (DFD):

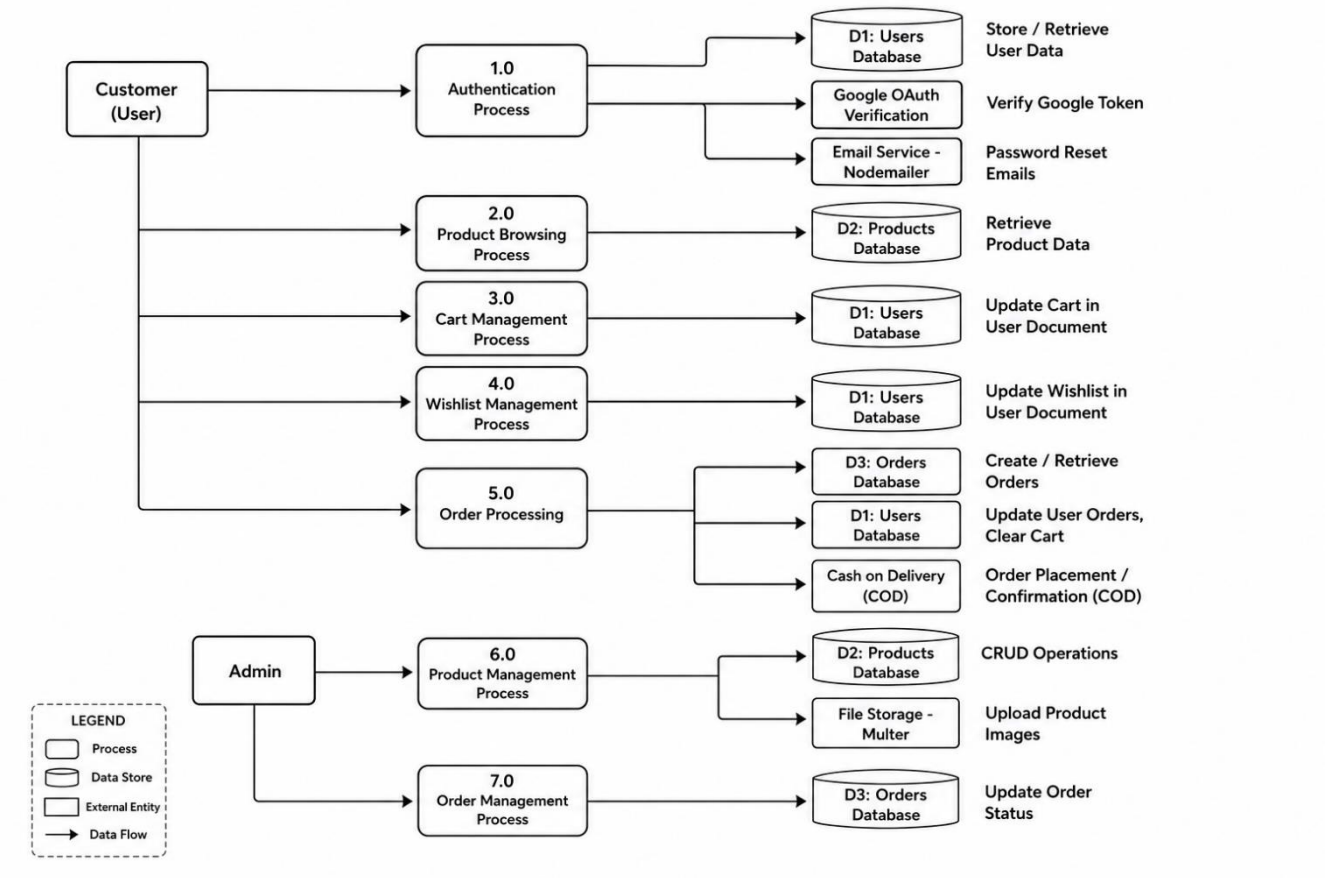
- **Level 0 DFD (Context Diagram)**



- The Level 0 DFD provides a high-level overview of the Style Hub e-commerce system. It represents the complete system as a single process and shows the flow of information between the system and external entities such as the Customer, Admin, Gmail SMTP, and Google OAuth. The Customer interacts with the system for registration, login, product browsing, cart and Wishlist operations, and order placement.

- The admin interacts with the Style Hub system to manage products and customer orders and receives the required dashboard information. The diagram also represents external services used by the system, such as Google OAuth for authentication and Gmail SMTP for sending password-reset emails. Overall, the Level 0 DFD gives a simple overview of how external entities communicate with the main system.

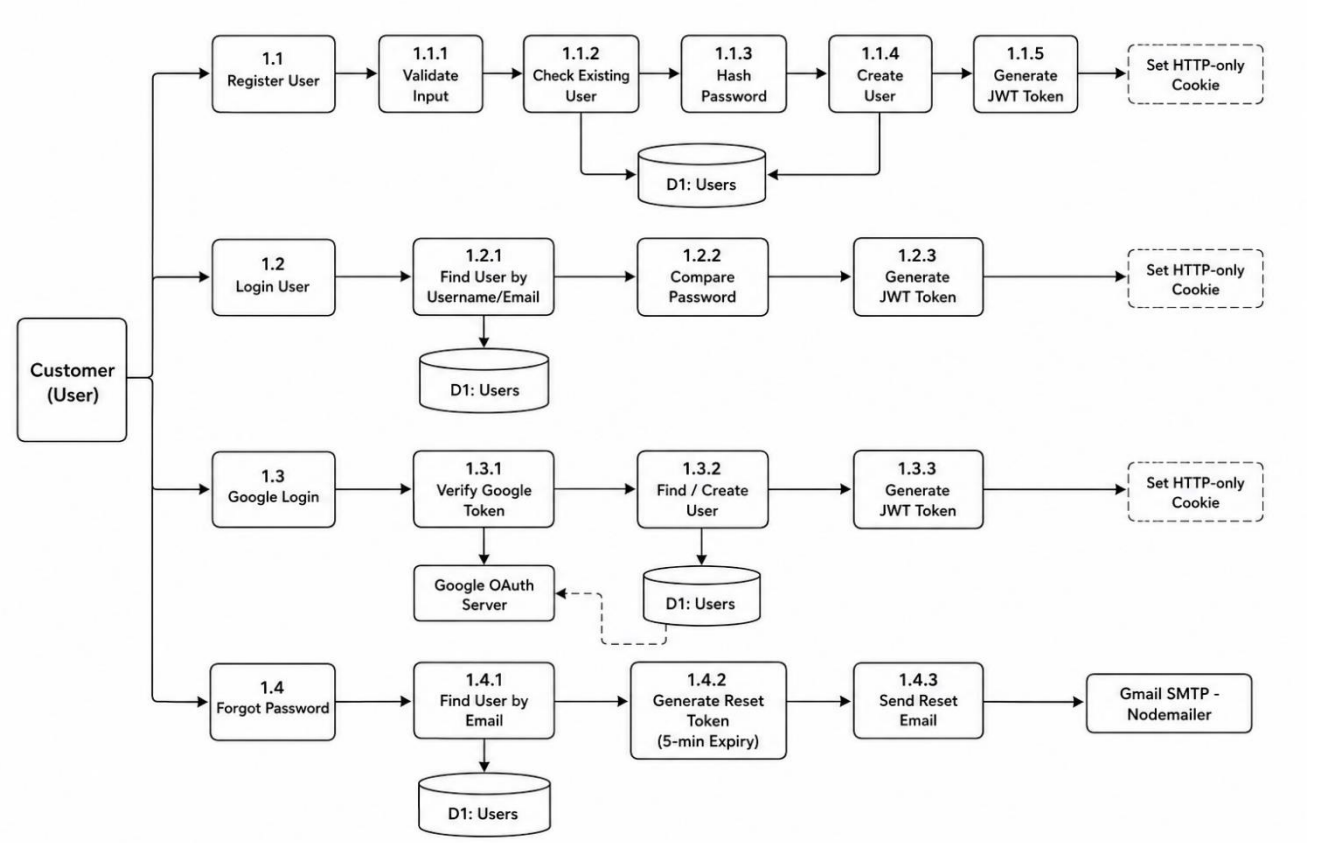
- **Level 1 DFD**



- The Level 1 DFD provides a detailed representation of the internal processes of the Style Hub system. The main processes include Authentication, Product Browsing, Cart Management, Wishlist Management, Order Processing, Product Management, and Order Management. Each process handles a specific part of the system and exchanges data with the required databases and external services.

- The Customer mainly interacts with authentication, product browsing, cart, Wishlist, and order processes, while the admin manages products and customer orders. The system uses User DB for user information, Product DB for product and stock details, and Order DB for order records and status. This diagram helps to understand the detailed flow of data between system processes, users, administrators, and databases.

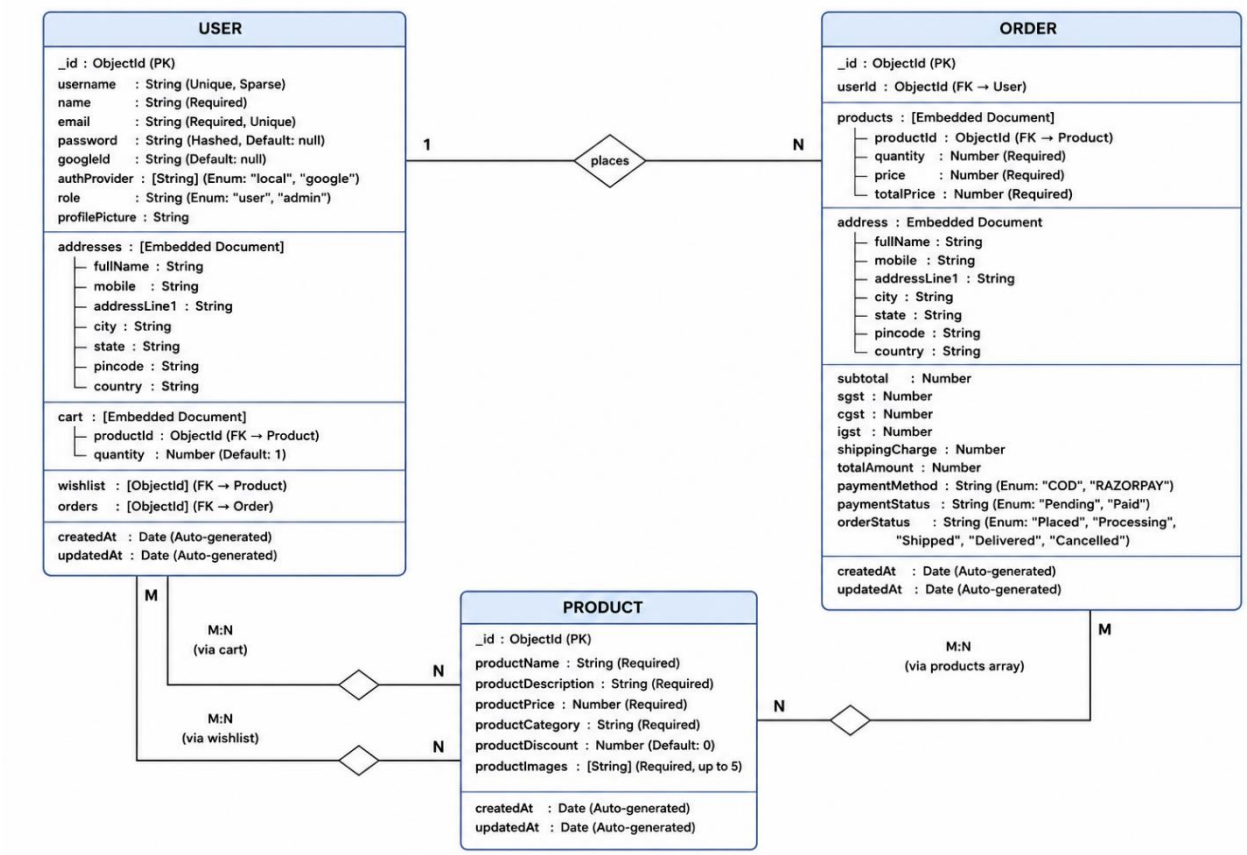
- **Level 2 DFD — Authentication Process**



- The Level 2 DFD shows the detailed flow of the Authentication Process (1.0). The Customer interacts with four sub-processes: Register User (1.1), Login User (1.2), Google Login (1.3), and Forgot Password (1.4). During registration, user input is validated, existing users are checked, the password is hashed, a new user is created, and a JWT token is generated.

- During login, the system finds the user by username/email, compares the password, and generates a JWT token. Google Login verifies the Google token, finds or creates the user, and generates a JWT token. In Forgot Password, the user is found by email, a 5-minute reset token is generated, and the reset email is sent through Gmail SMTP/Node mailer. The user information is stored in the D1: Users Database, and authentication tokens are set using an HTTP-only cookie.

4.2 E-R Diagram (Entity-Relationship Diagram):



1. User Model:

- The User Model stores all the important information related to customers and administrators of the Style Hub system. It contains basic details such as username, name, email, password, authentication provider, role, and profile picture. It also maintains the user's saved addresses, shopping cart, Wishlist, and order references. This model helps the system identify users, manage their accounts, and provide personalized shopping features.

- The User Model is connected with the Product and Order models through references. A user can add multiple products to the cart or Wishlist and can place multiple orders. The cart stores the selected product and its quantity, while the Wishlist stores products saved by the user for future use. This model therefore manages the complete customer-related data required for browsing, cart management, and order placement.

2. Product Model:

- The Product Model stores information about all products available on the Style Hub website. It contains details such as product name, description, price, category, discount, product images, and creation or update dates. This information is used to display products in the product catalog and helps customers search, browse, and filter products according to their requirements.

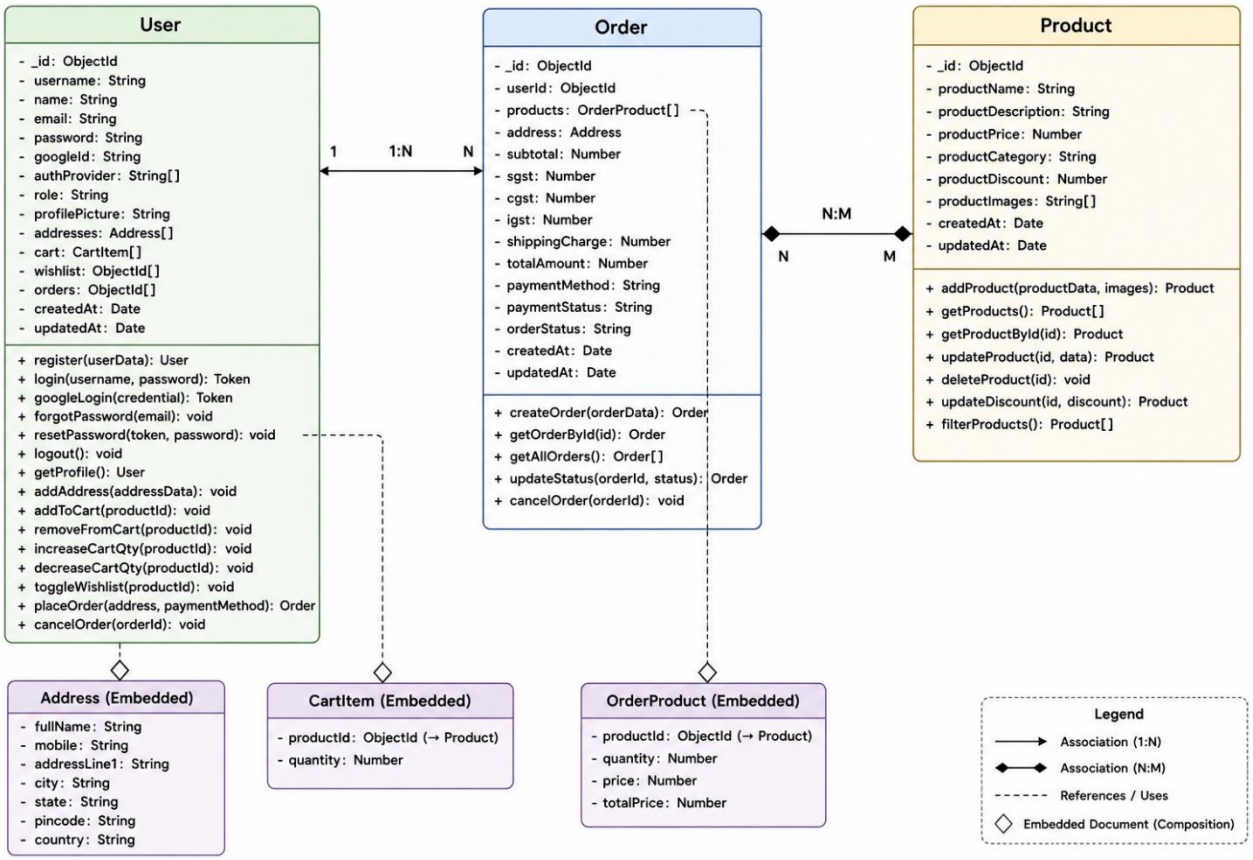
- The Product Model is also used by the admin for managing the product catalog. The administrator can add, update, delete, and manage products and categories through the admin panel. Products can be added to a user's cart or Wishlist and can also become part of an order when the customer completes the checkout process.

3. Order Model:

- The Order Model stores the complete information about customer orders. It contains the user ID, ordered products, product quantity, price, total price, delivery address, subtotal, taxes, shipping charge, total amount, payment method, payment status, and order status. This allows the system to maintain a complete record of each transaction and its current status.

- When a customer completes checkout, the system creates an order record and clears the active cart. The order information is then available for administration and order tracking. The admin can view customer orders and manage their status, while the customer receives confirmation after successful order placement.

4.3 Class Diagram:



1. User Class:

- The User Class manages all customer and administrator-related information in the Style Hub system. It stores details such as username, name, email, password, Google ID, authentication provider, role, profile picture, addresses, cart, Wishlist, and orders.

- It also provides operations for registration, login, Google login, password recovery, profile management, address management, cart management, Wishlist management, and order placement/cancellation. This class is mainly responsible for handling user accounts and customer activities.

2. Order Class:

- The Order Class manages all information related to customer orders. It stores the user ID, ordered products, delivery address, subtotal, SGST, CGST, IGST, shipping charge, total amount, payment method, payment status, and order status.

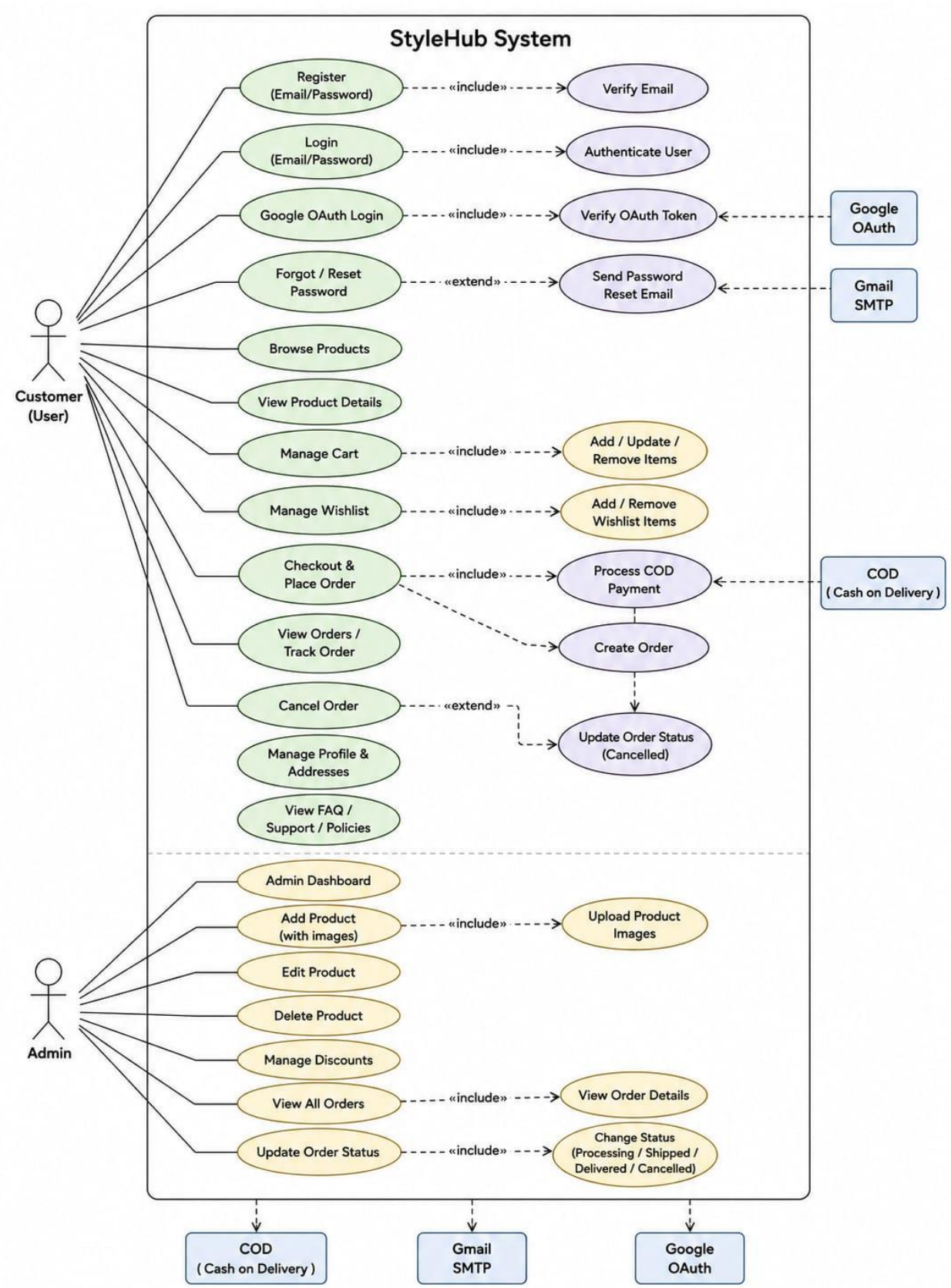
- It provides operations such as creating an order, retrieving an order, getting all orders, updating order status, and cancelling an order. The Order Class is connected with the User Class because a user can place multiple orders, and it is connected with the Product Class because an order can contain multiple products.

3. Product Class:

- The Product Class manages the products available in the Style Hub system. It stores product name, description, price, category, discount, product images, and creation/update dates.

- It provides operations for adding, retrieving, updating, deleting, filtering products, and updating product discounts. Products can be added to the user's cart or wishlist and can also be included in orders. The Product Class is therefore an important part of the product browsing and product management functionality.

4.4 Use Case Diagram:



1. Customer (User):

- **Authentication & Profile Management:**
  Users can register an account using an email address and password, which triggers an automated email verification step. Alternatively, users can authenticate seamlessly via Google OAuth, which verifies an identity token provided by Google's external authentication service. For account recovery, users can initiate a password reset request; this extends to sending a reset email via an external Gmail SMTP server. Furthermore, customers can manage their profile details and saved shipping addresses within the system.

- **Product Discovery and Cart Operations:**
  Customers can browse the full product catalog and view comprehensive product details. To manage intended purchases, users can interact with the shopping cart to add, update, or remove items. Similarly, users can maintain a personal Wishlist by adding or removing desired items.

- **Order Management and Checkout:**
  During checkout, the customer places an order, which includes processing payment via Cash on Delivery (COD) and generating a unique order record. Customers can view their order history and track the current shipment status. Additionally, the system allows customers to cancel active orders, which automatically extends to updating the order status to "Cancelled."

2. Admin:

- **Dashboard and Product Catalog Management:**
  Administrators access the main Admin Dashboard to monitor platform activity. They hold full Control over the product catalog, including adding new products (which includes uploading product image files), editing existing product details, and deleting obsolete items. Admins can also create and manage promotional discounts applied across the platform.

- **Order Lifecycle Management:**
  Administrators have the authority to view all placed orders and inspect detailed order information. To keep customers informed, admins update the lifecycle status of orders through various stages, including Processing, Shipped, Delivered, or Cancelled.

3. External System Boundaries:

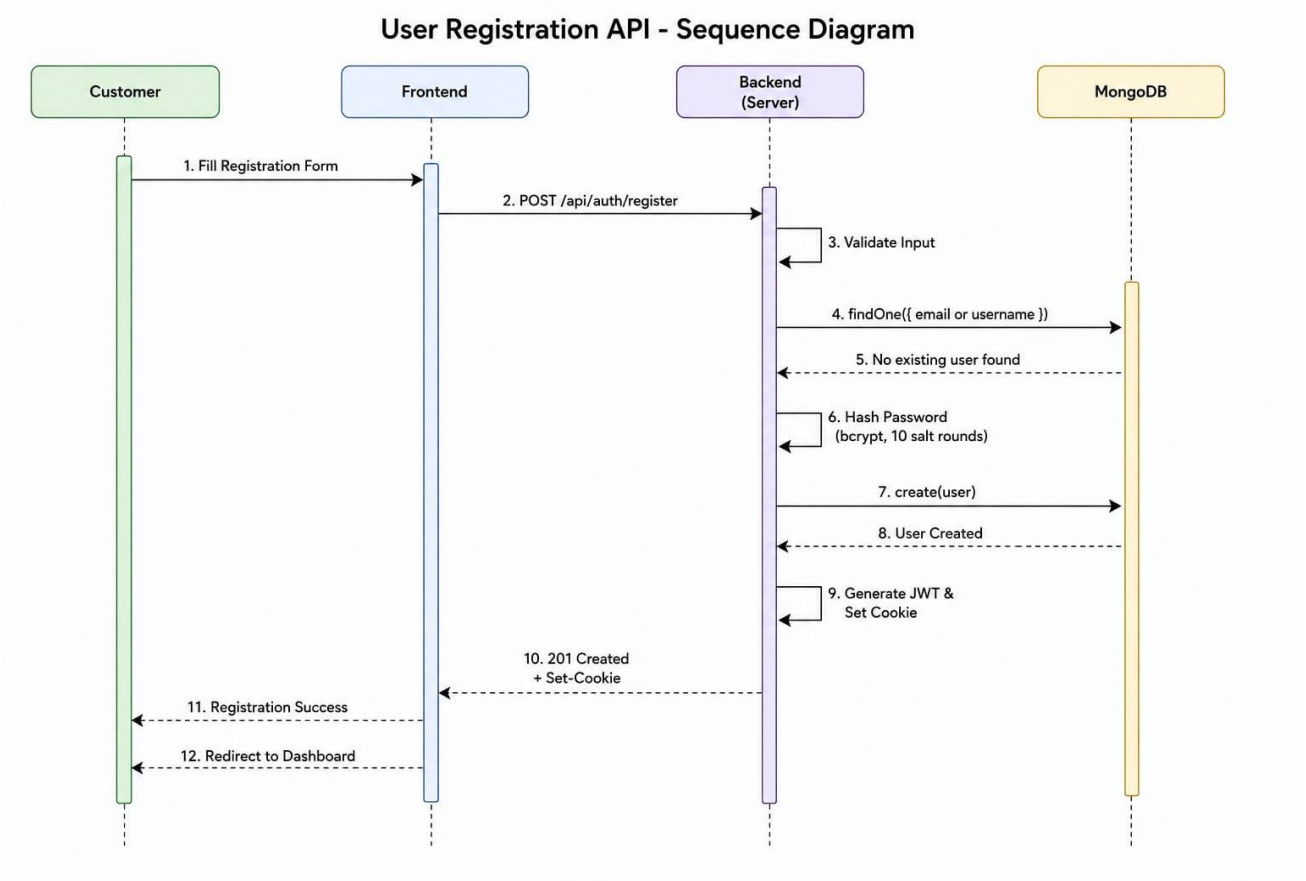
- **Google OAuth:**
  Handles secure identity verification during third-party sign-ins.

- **Gmail SMTP:**
  Serves as the mail transport agent responsible for dispatching transactional emails such as password resets.

- **Cash on Delivery (COD):**
  Functions as the primary payment gateway model for verifying and processing offline payments upon order delivery.

4.5 Sequence Diagram:

Sequence Diagram 1: User Registration:



1. **Fill Registration Form:**
   The customer enters the required details such as username, name, email, and password in the registration form.

2. **Send Registration Request:**
   The frontend sends the registration data to the backend using the POST /api/auth/register API.

3. **Validate Input:**
   The backend validates the submitted registration details to ensure that all required data is valid.

4. **Check Existing User:**
   The backend checks MongoDB using findOne ({email or username}) to determine whether the email or username already exists**.**

5. No Existing User Found:

If no existing user is found, the registration process continues.

6. Hash Password:

The backend securely hashes the user's password using bcrypt with 10 salt rounds before storing it in the database.

7. Create User:

The backend creates a new user record and sends it to MongoDB using the create(user) operation.

8. User Created:

MongoDB stores the new user information and returns a successful user-created response to the backend.

9. Generate JWT and Set Cookie:

After successful registration, the backend generates a JWT (JSON Web Token) and sets it in a secure cookie for user authentication.

10. Return Success Response:

The backend sends a 201 Created response along with the authentication cookie to the frontend.

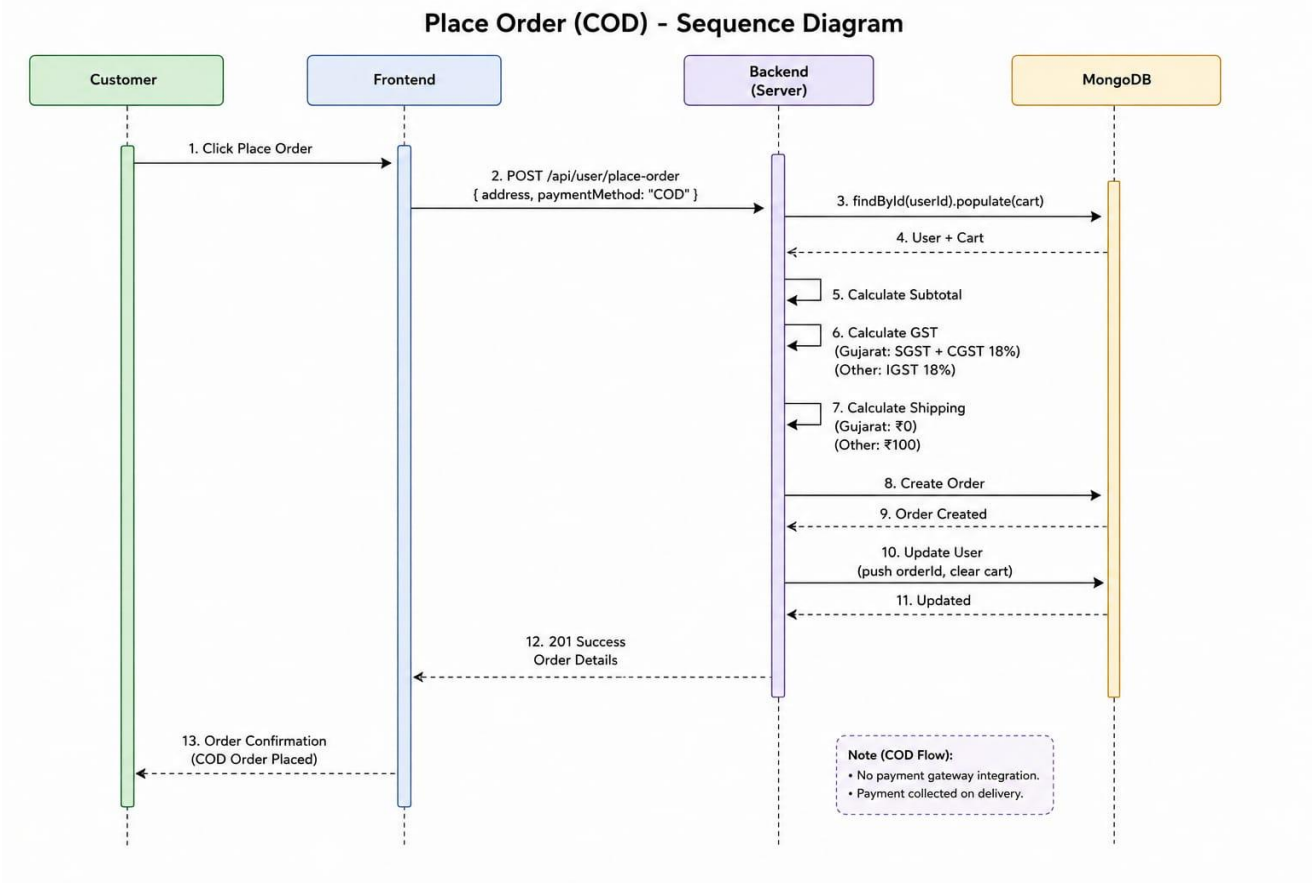
11. Registration Success:

The frontend receives the successful response and displays a registration success message to the customer.

12. Redirect to Dashboard:

After successful registration, the customer is redirected to the Dashboard for further access to the system.

Sequence Diagram 2: Place Order



1. **Order Initiation:**
   The Customer clicks the Place Order button on the user interface.

2. **API Request:**
   The Frontend sends an HTTP POST request to /api/user/place-order containing payload details: {address, paymentMethod: "COD”}.

3. **Fetch User & Cart Data:**
   The Backend queries MongoDB using findById(userId). populate(cart) to retrieve the current user's profile along with their cart items.

4. **Data Retrieval:**
   MongoDB returns the combined User + Cart dataset to the Backend

5. **Subtotal Calculation:**
   The Backend calculates the subtotal amount based on the items present in the cart.

6. **GST Calculation:**
   The Backend calculates applicable taxes based on location logic:

- **Within Gujarat:** Applies intra-state tax rate (SGST + CGST 18%).
- **Outside Gujarat (Other):** Applies inter-state tax rate (IGST 18%).

7. Shipping Charge Calculation:

The Backend evaluates shipping charges based on location rules:

- **Gujarat:** Free shipping (₹0).
- **Outside Gujarat (Other):** Flat shipping charge (₹100).

8. Create Order Entity:

The Backend sends a request to MongoDB to store the newly generated order record.

9. Persisted:

MongoDB confirms the order creation by returning Order Created.

10. Update User Record:

The Backend sends an update request to MongoDB to push the new orderId into the user's order history and clear the active cart.

11. Update Confirmation:

MongoDB returns an Updated acknowledgment back to the Backend.

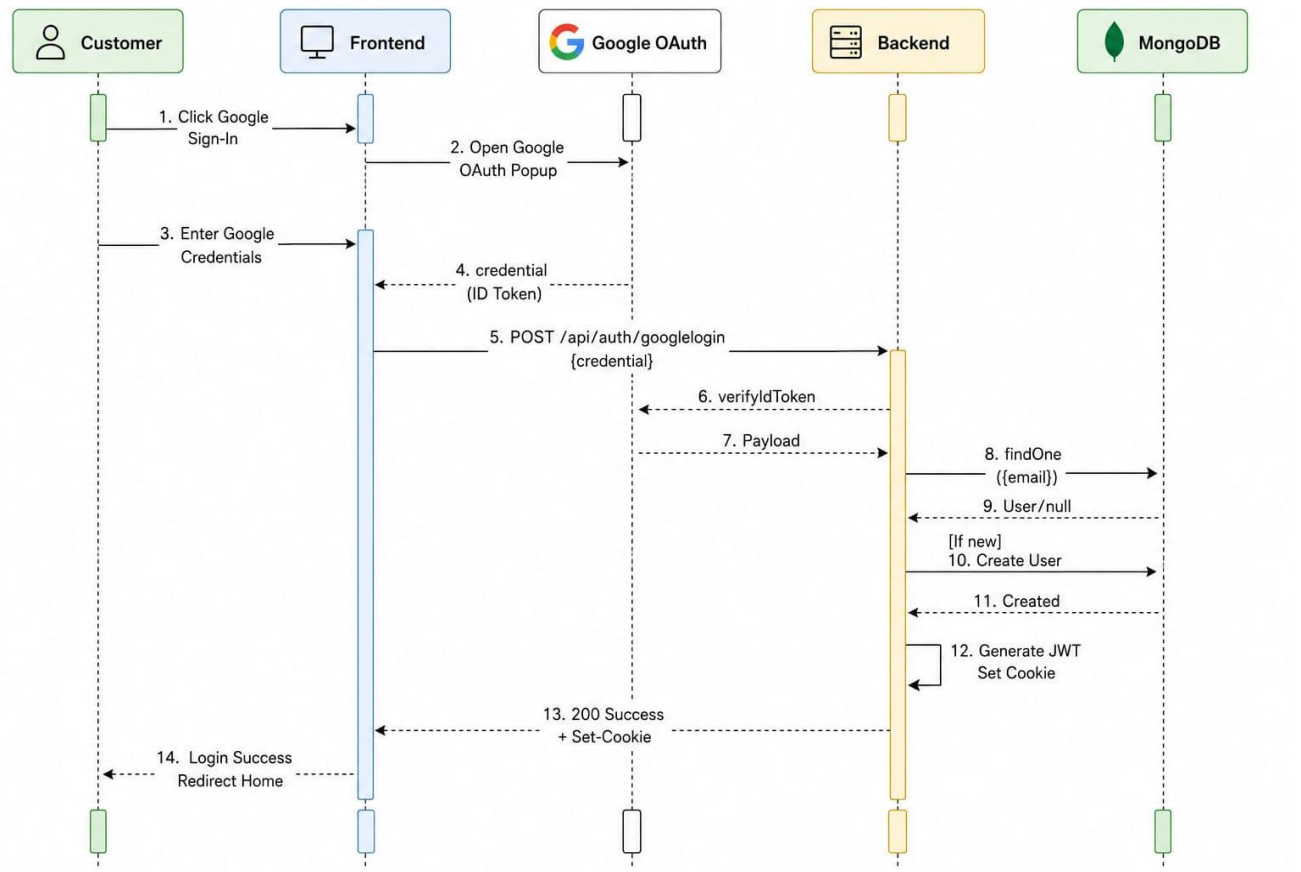
12. API Response:

The Backend sends a 201 Success HTTP status code along with the complete Order Details to the Frontend.

13. Order Confirmation:

The Frontend displays an Order Confirmation (COD Order Placed) message to the Customer.

Sequence Diagram 3: Google OAuth Login



1. **User Action:**
   The Customer clicks Google Sign-In on the application.

2. **OAuth Request:**
   The Frontend opens the Google OAuth Popup window.

3. **Credentials Entry:**
   The Customer submits their Google account credentials in the popup window.

4. **Token Generation:**
   Google OAuth verifies the credentials and returns a credential (ID Token) back to the Frontend.

5. **Backend Authentication Request:**
   The Frontend sends an HTTP POST request to /api/auth/googlelogin with {credential} payload.

6. **Token Verification Request:**
   The Backend sends a verifyIdToken request to Google OAuth to validate the received token.

7. Payload Retrieval:
Google OAuth verifies the token and sends back the user's decoded profile Payload.

8. User Query:
The Backend checks MongoDB for existing account details using findOne({email}).

9. Database Response:
MongoDB returns the matching User record or null if the user is not found.

10. Account Creation (If New User):
If the user does not exist (null), the Backend issues a Create User command to MongoDB.

11. Creation Confirmation:
MongoDB returns a Created status confirmation.

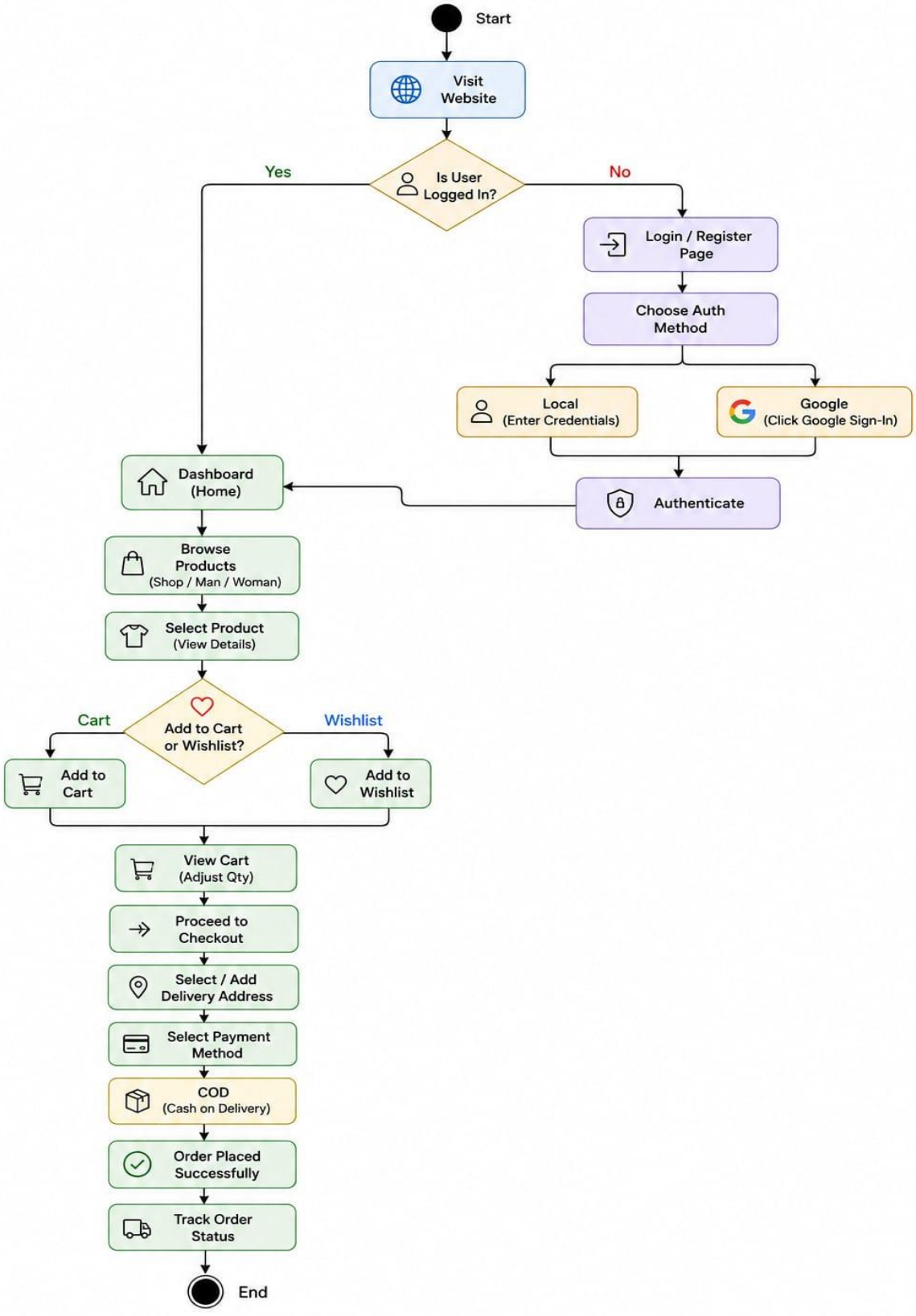
12. Session Token Generation:
The Backend executes internal processing to Generate JWT and prepare a Set-Cookie header.

13. API Response:
The Backend returns a 200 Success status code along with the session cookie (Set-Cookie) to the Frontend.

14. Login Completion:
The Frontend receives the confirmation and triggers Login Success / Redirect Home for the Customer.

4.6 Activity Diagram:

Activity Diagram 1: User Shopping Flow



1. Entry & Authentication Phase:

- **Start:** User accesses the platform by visiting the website.
- **Authentication Decision:** The system checks if the user is logged in:
    - **Yes:** User routes directly to the Dashboard (Home) page.
    - **No:** User redirects to the Login / Register Page to select an authentication method:
        - **Local:** User enters standard credentials.
        - **Google:** User selects Google Sign-In.

2. Product Discovery & Selection Phase:

- **Browse Products:** User explores catalog items across categories (Shop / Man / Woman).
- **Select Product:** User views detailed product information (View Details).
- **Cart/Wishlist Decision:** User chooses how to store the selected item:
    - **Cart:** Adds item via Add to Cart.
    - **Wishlist:** Saves item via Add to Wishlist.

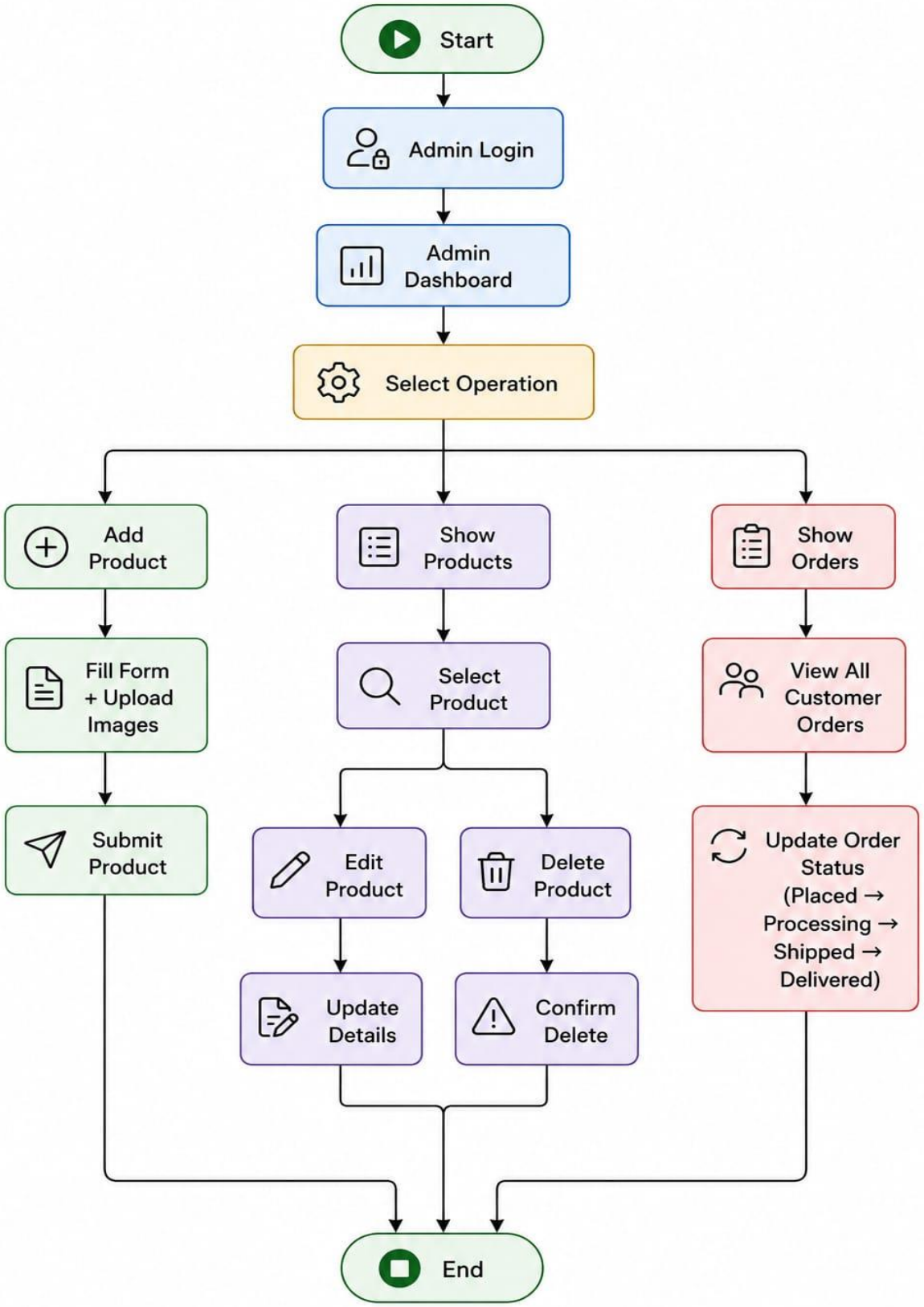
3. Checkout & Payment Phase:

- **View Cart:** User reviews selected items and adjusts quantities (Adjust Qty).
- **Proceed to Checkout:** User initiates the checkout workflow.
- **Delivery Setup:** User chooses an existing address or inputs a new one (Select / Add Delivery Address).
- **Payment Selection:** User chooses their preferred payment method (Select Payment Method).
- **COD Execution:** Order processes via COD (Cash on Delivery).

4. Order Completion & Exit Phase:

- **Order Confirmation:** System displays Order Placed Successfully.
- **Order Tracking:** User monitors fulfillment via Track Order Status.
- **End:** Activity workflow finishes.

Activity Diagram 2: Admin Product Management Flow



1. Authentication & Navigation:

- **Start:** The administrator initiates the management workflow.
- **Admin Login:** The system verifies administrator credentials.
- **Admin Dashboard:** Access is granted to the centralized management dashboard.
- **Select Operation:** The admin selects one of three primary operational branches: catalog creation, catalog management, or order processing.

2. Branch A: Product Creation (Add Product):

- **Add Product:** The admin selects the option to create a new catalog item.
- **Fill Form + Upload Images:** The admin inputs item specifications and uploads associated product imagery.
- **Submit Product:** The system validates and saves the new product record.

3. Branch B: Product Catalog Management (Show Products):

- **Show Products:** The system displays the existing catalog list.
- **Select Product:** The admin chooses a specific item to modify or remove.
- **Operation Branching:**
    - **Edit Workflow:** Admin selects Edit Product → submits modifications via Update Details.
    - **Delete Workflow:** Admin selects Delete Product → verifies removal via Confirm Delete.

4. Branch C: Order Management (Show Orders):

- **Show Orders:** The admin opens the order management section.
- **View All Customer Orders:** The system displays all placed customer orders.
- **Update Order Status:** The admin modifies order fulfilment stages along the lifecycle: Placed → Processing → Shipped → Delivered.

5. Flow Termination:

- **End:** All operational paths converge to complete the admin activity process.

4.7 Data Dictionary (Database):

Collection 1: Users

Field Name	Data Type	Constraints	Default	Description
_id	ObjectId	Primary Key, Auto-generated	—	Unique identifier for each user
username	String	Unique, Sparse Index	—	User's login username
name	String	Required	—	User's display name
email	String	Required, Unique	—	User's email address for communication and login
password	String	—	null	Bcrypt-hashed password
googleId	String	—	null	Google account ID for Google OAuth users
authProvider	String	Enum: local, google	local	Authentication method used by the user
role	String	Enum: user, admin	user	User role for access control
profilePicture	String	—	""	URL or path to user's profile picture
addresses	Array of Objects	Embedded Array	[]	Array of saved delivery addresses
addresses.fullName	String	—	—	Full name of the address recipient
addresses.mobile	String	—	—	Mobile number for delivery contact
addresses.addressLine1	String	—	—	Street address or building details
addresses.city	String	—	—	City name

addresses.state	String	—	—	State name used for GST calculation
addresses.pincode	String	—	—	PIN code of the delivery address
addresses.country	String	—	India	Country of the delivery address
cart	Array of Objects	Embedded Array	[]	Array of products in the user's cart
cart.productId	ObjectId	Reference → Product	—	References the Product collection
cart.quantity	Number	—	1	Quantity of the product in the cart
wishlist	Array of ObjectIds	Reference → Product	[]	Products saved to the wishlist
orders	Array of ObjectIds	Reference → Order	[]	Orders placed by the user
createdAt	Date	Auto-generated	—	Date and time when the user was created
updatedAt	Date	Auto-generated	—	Date and time when the user was last updated

Collection 2: Products

Field Name	Data Type	Constraints	Default	Default
_id	ObjectId	Primary Key, Auto-generated	—	Unique identifier for each product
productName	String	Required	—	Name or title of the product
productDescription	String	Required	—	Detailed description of the product
productPrice	Number	Required	—	Price of the product in INR (₹)
productCategory	String	Required	—	Category of the product, such as Men, Women, or Accessories
productDiscount	Number	Optional	0	Discount percentage on the product (0–100)
productImages	Array of Strings	Required, Minimum 1, Maximum 5	—	Array of image filenames uploaded using Multer
createdAt	Date	Auto-generated	—	Date and time when the product was created
updatedAt	Date	Auto-generated	—	Date and time when the product was last updated

Collection 3: Orders

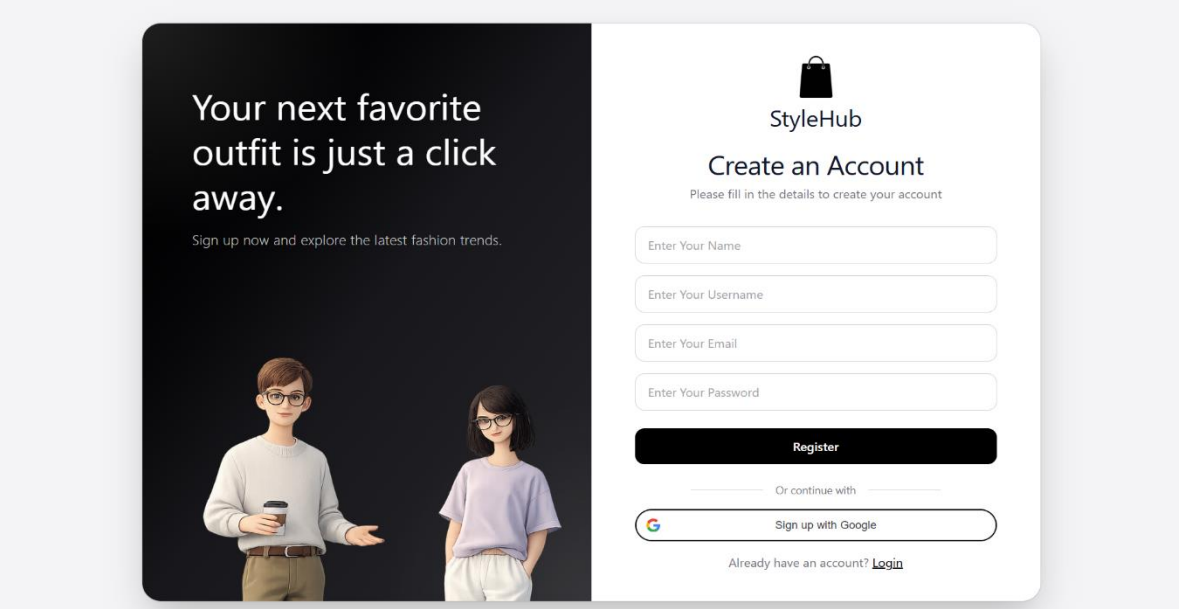
Field Name	Data Type	Constraints	Default	Description
_id	ObjectId	Primary Key, Auto-generated	—	Unique identifier for each order
userId	Array of Objects	Reference → User	—	Foreign key referencing the User who placed the order
products	ObjectId	Embedded Array	—	Array of products included in this order
products.productId	ObjectId	Reference → Product	—	Foreign key referencing the Product
products.quantity	Number	Required	—	Quantity of the product ordered
products.price	Number	Required		Unit price of the product at the time of order
products.totalPrice	Number	Required	—	Total price calculated as price × quantity
address	Object	Embedded Document	—	Delivery address for this order
address.fullName	String	—	—	Recipient's full name
address.mobile	String	—	—	Recipient's mobile number
address.addressLine1	String	—	—	Street address
address.city	String	—	—	Delivery city
address.state	String	—	—	Delivery state
address.pincode	String	—	—	Delivery PIN code
address.country	String	—	—	Delivery country

subtotal	Number	—	—	Sum of all product total prices before tax
sgst	Number	—	—	State GST (9% of subtotal for Gujarat)
cgst	Number	—	—	Central GST (9% of subtotal for Gujarat)
igst	Number	—	—	Integrated GST (18% of subtotal for non-Gujarat)
shippingCharge	Number	—	—	Shipping fee (₹0 for Gujarat, ₹100 for other states)
shippingCharge	Number	—	—	Final amount = subtotal + GST + shipping
paymentMethod	String	Enum: COD	—	Payment method chosen by the customer
paymentStatus	String	Enum: Pending, Paid	Pending	Current payment status of the order
orderStatus	String	Enum: Placed, Processing, Shipped, Delivered, Cancelled	Placed	Current order fulfillment status
createdAt	Date	Auto-generated (timestamps)	—	Timestamp when the order was placed
updatedAt	Date	Auto-generated (timestamps)	—	Timestamp when the order was last updated

5. SCREEN DESIGN

5.1 Input Design:

Register.jsx:

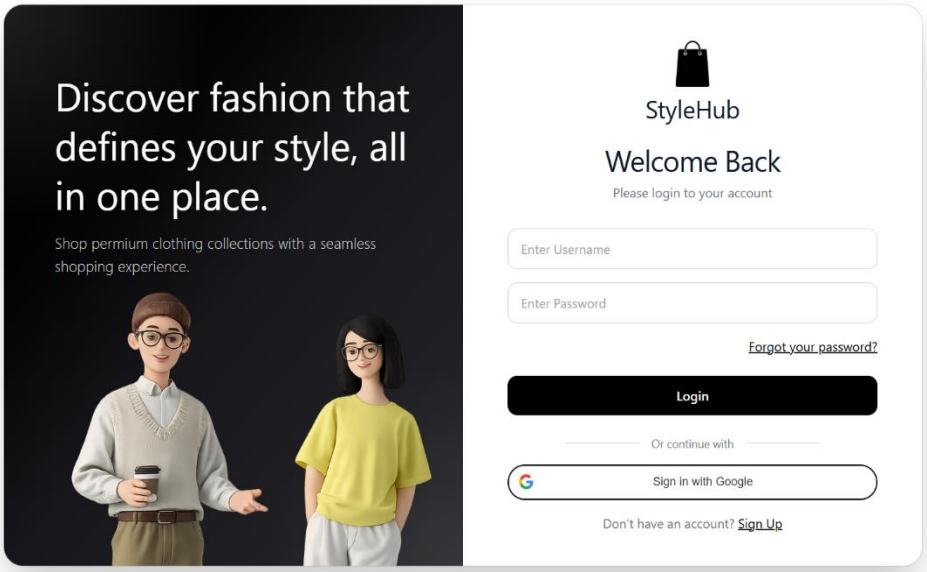


Key UI Components & Layout:

- **Left Banner (Branding & Promotion):**
  - **Tagline:** "Your next favorite outfit is just a click away."
  - **Sub-text:** "Sign up now and explore the latest fashion trends."
  - **Visual Elements:** Engaging 3D character illustrations aligned with the fashion and shopping theme.

- **Right Form Section (User Registration):**
  - **Brand Logo & Header:** StyleHub brand logo accompanied by the "Create an Account" header.
  - **Input Fields:**
    - **Enter Your Name:** Input field for the user's full name.
    - **Enter Your Username:** Input field to set a unique system username.
    - **Enter Your Email:** Input field for email address validation.
    - **Enter Your Password:** Input field for setting a secure account password.
  - **Action Buttons:**
    - **Register Button:** Primary Call-to-Action (CTA) button to submit form details and complete registration.
    - **Sign up with Google:** Third-party social login integration (OAuth 2.0).
  - **Navigation Link:** "Already have an account? Login" link directing existing users to the login page.

Login.jsx:

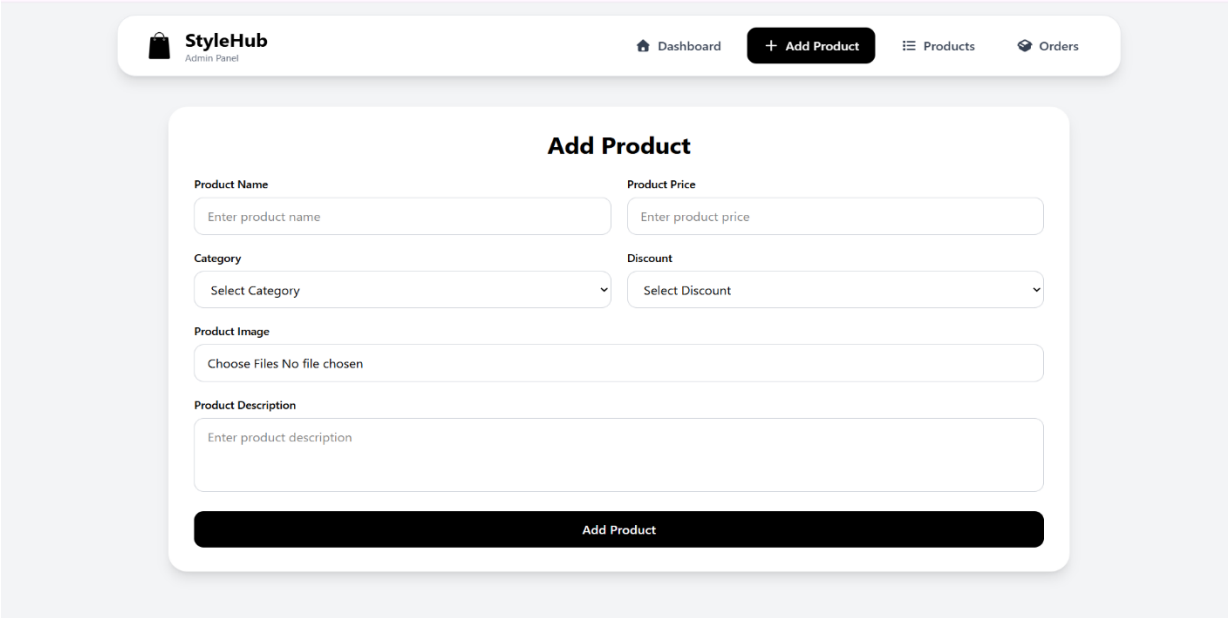


Key UI Components & Layout:

- **Left Banner (Branding & Value Proposition):**
  - **Tagline:** "Discover fashion that defines your style, all in one place."
  - **Sub-text:** "Shop premium clothing collections with a seamless shopping **experience."**
  - **Visual Elements:** 3D animated character illustrations reinforcing brand identity.

- **Right Form Section (User Authentication):**
  - **Brand Logo & Header:** StyleHub logo paired with the "Welcome Back" heading and secondary prompt "Please login to your account".
  - **Input Fields:**
    - **Enter Username:** Input field for the registered user's account name.
    - **Enter Password:** Input field for entering the secret password.
  - **Account Recovery:**
    - **Forgot your password? Link:** Allows users to initiate the password reset process if credentials are forgotten.
  - **Action Buttons:**
    - **Login Button:** Primary Call-to-Action (CTA) button to authenticate and access the dashboard/store.
    - **Sign in with Google:** Social login option via OAuth 2.0 for single-click authentication.
  - **Navigation Link:**
    - "Don't have an account? Sign Up" link directing new users directly to the registration page.

Addproduct.jsx (admin):



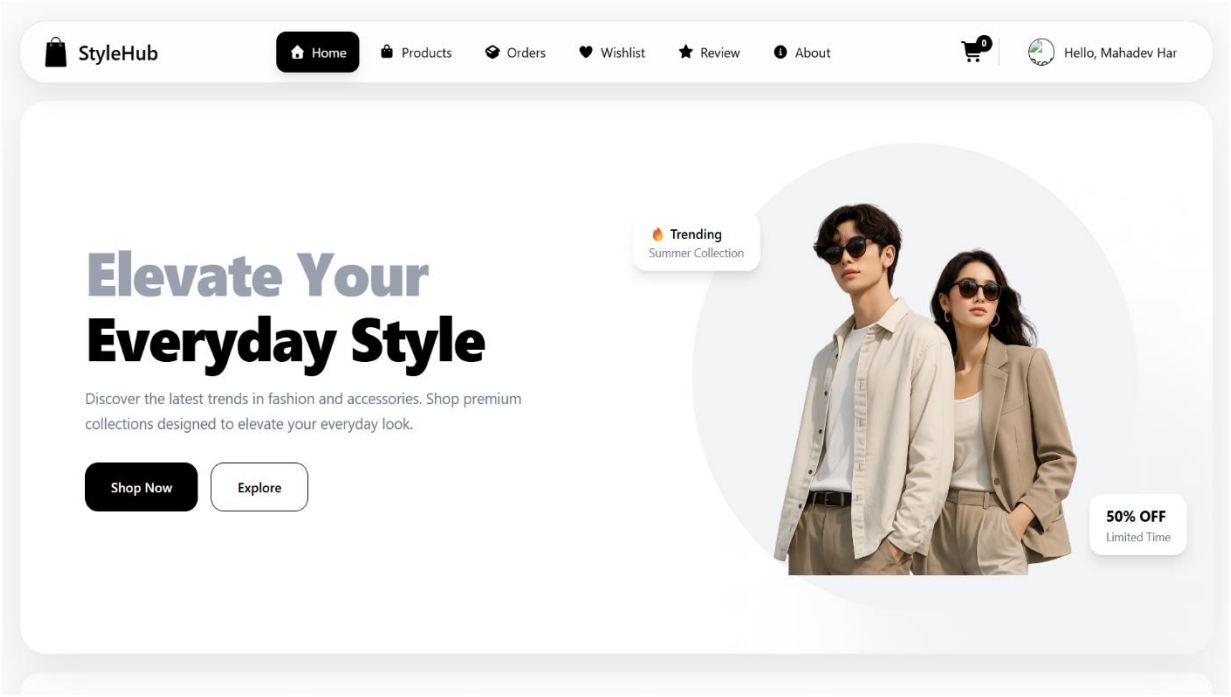
Key UI Components & Layout:

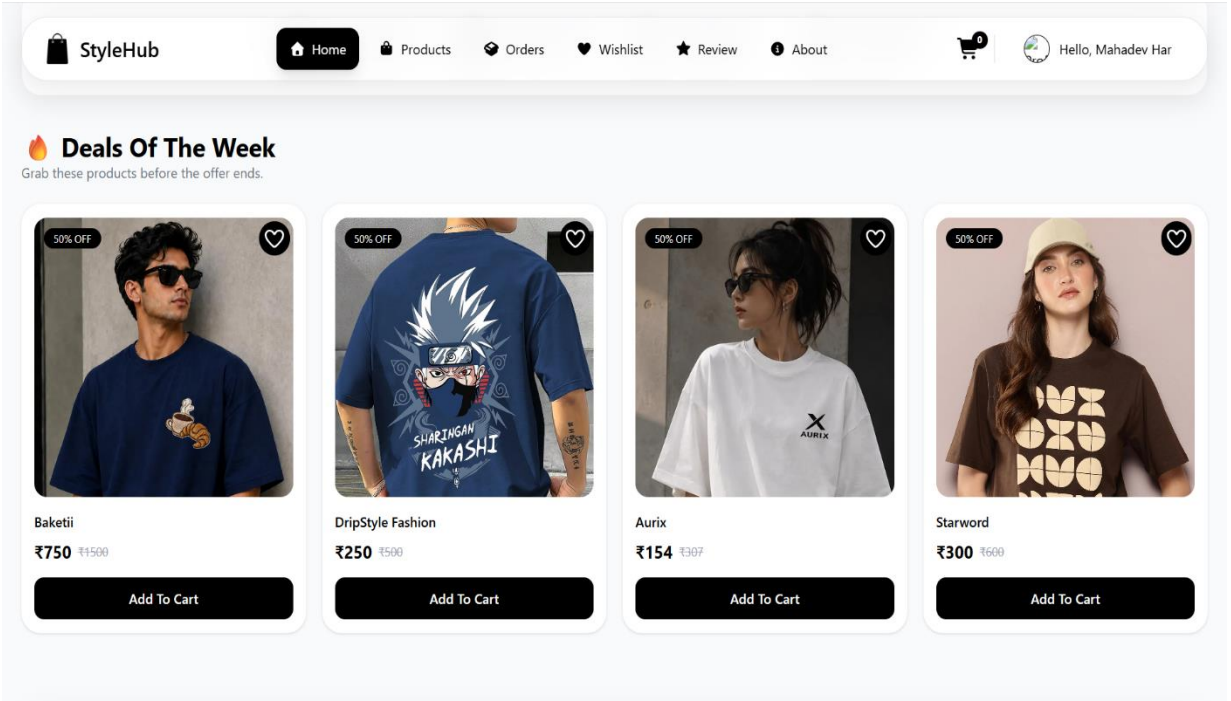
- **Admin Navigation Bar (Header):**
  - **Brand Identification:** Displays StyleHub logo with the "Admin Panel" sub-label.
  - **Navigation Tabs:** Direct routing tabs (Dashboard, Products, Orders) with the active tab + Add Product highlighted in a black pill button.

- **Form Container & Layout (2-Column Grid Layout):**
  - **Page Title:** "Add Product" header centered at the top of the form panel.
  - **Form Input Fields:**
    - **Product Name:** Text input box with placeholder "Enter product name".
    - **Product Price:** Numerical input box with placeholder "Enter product price".
    - **Category:** Dropdown selection menu ("Select Category") to assign items to specific catalog sections (e.g., Man, Woman).
    - **Discount:** Dropdown menu ("Select Discount") to apply promotional percentage discounts.
    - **Product Image:** File upload input ("Choose Files No file chosen") to attach high-resolution product photography.
    - **Product Description:** Multi-line text area field with placeholder "Enter product description" for detailed item specifications.
  - **Submit Action Control:**
    - **"Add Product" Button:** Full-width black Call-to-Action (CTA) button at the base of the form panel to submit data to the backend.

5.2 Output Design:

Home.jsx:





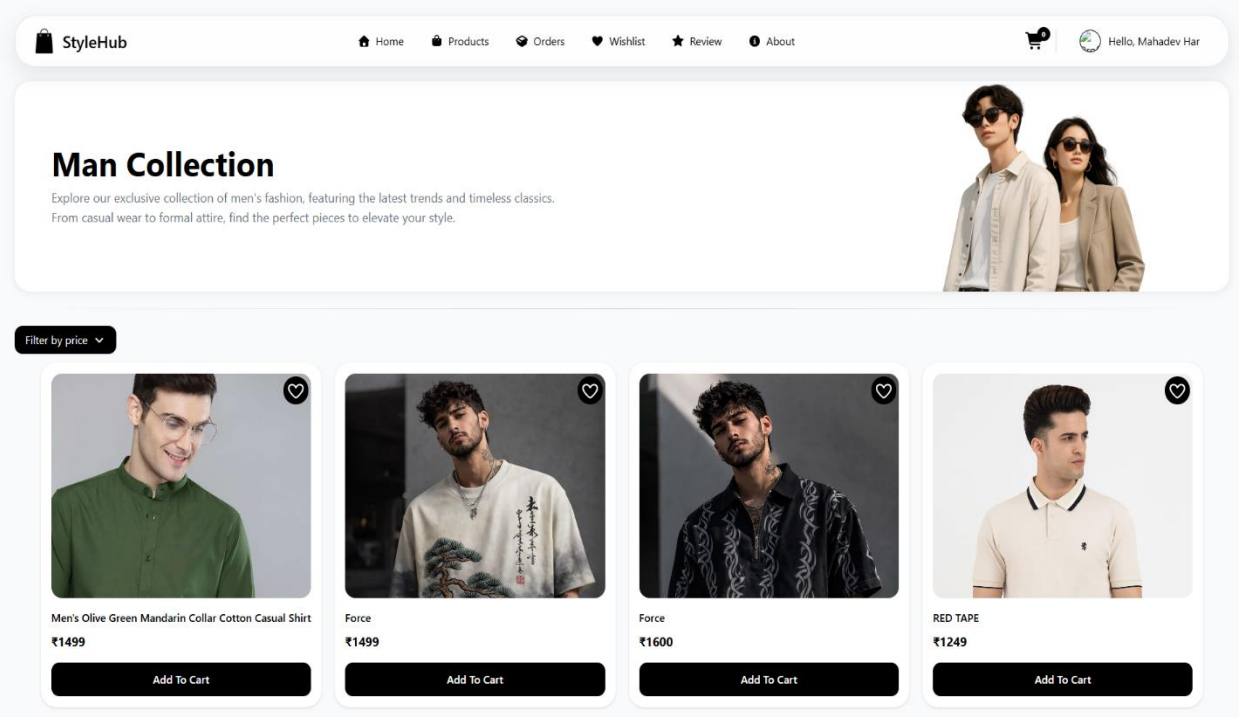
Key UI Components & Layout:

- **Navigation Bar (Header Component):**
  - **Brand Logo:** Displays the StyleHub logo on the far left.
  - **Navigation Links:** Quick links for seamless routing
    - **Home:** Highlights the current active landing view.
    - **Products / Orders / Wishlist / Review / About:** Direct links to catalog, order history, saved items, reviews, and company details.
  - **Utility Section:**
    - **Shopping Cart Icon:** Dynamic counter displaying items added (e.g., 0).
    - **User Profile Indicator:** Displays a personalized greeting showing the active user account ("Hello, Mahadev Har").

- **Hero Banner Section:**
  - **Primary Headline:** "Elevate Your Everyday Style" paired with descriptive subtext encouraging exploration.
  - **Call-to-Action (CTA) Buttons:**
    - **Shop Now:** Primary CTA redirecting users directly to the shopping catalog.
    - **Explore:** Secondary CTA for browsing collections.
  - **Promotional Badges & Imagery:**
    - High-quality promotional model graphics.
    - Interactive floating tags such as "🔥 Trending - Summer Collection" and "50% OFF - Limited Time".

- **Deals Of The Week (Product Grid):**
  - **Section Header:** Prominently highlights active promotional discounts ("Grab these products before the offer ends.").
  - **Product Cards (4-Column Layout):**
    - **Product Images & Badges:** Displays high-resolution photos along with a top-left discount badge ("50% OFF") and a top-right wishlist (heart) icon.
    - **Product Details:** Shows the brand/item title (e.g., Baketii, DripStyle Fashion, Aurix, Starword).
    - **Pricing Engine:** Features dynamic pricing displaying original cross-out prices alongside discounted rates (e.g., ₹250 original ₹500).
    - **Interactive CTA:** Each product card includes an individual "Add to Cart" button for immediate purchasing.

Man.jsx



Key UI Components & Layout:

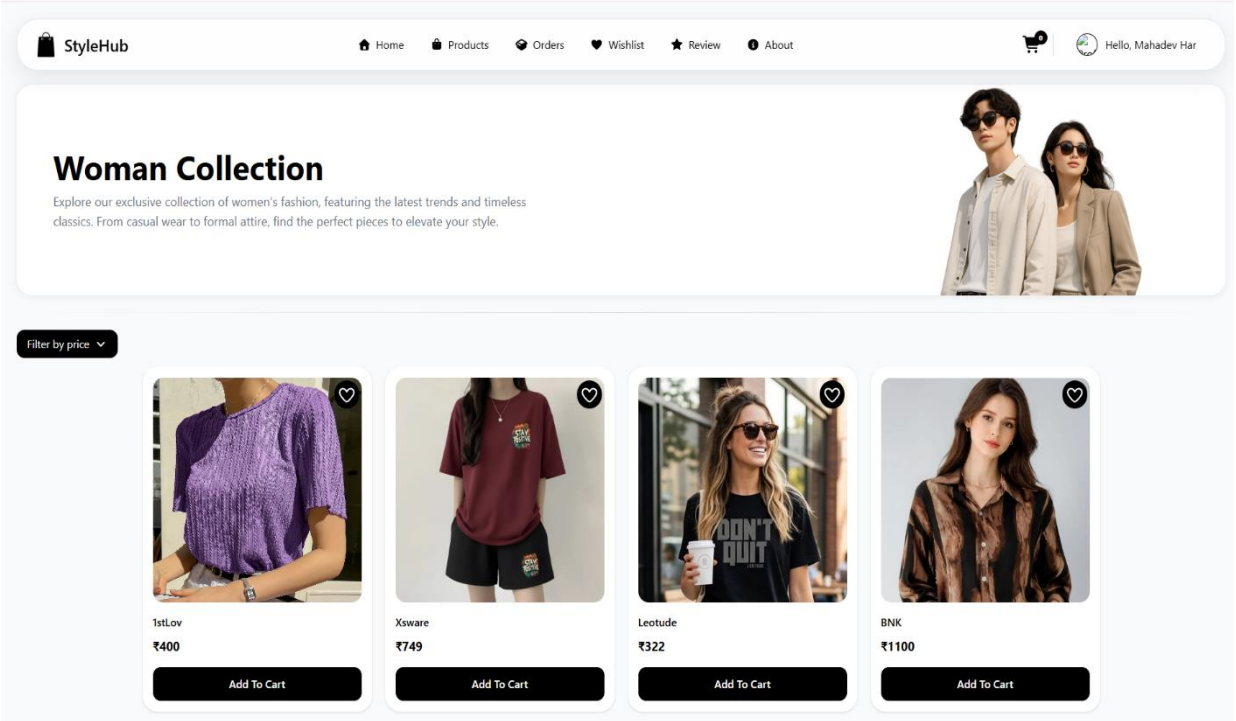
- **Navigation Bar (Header):**
  - Retains consistent global navigation options (Home, Products, Orders, Wishlist, Review, About).
  - Includes cart item counter (0) and logged-in user profile header ("Hello, Mahadev Har").

- **Category Header Banner:**
  - **Title:** "Man Collection"
  - **Description:** "Explore our exclusive collection of men's fashion, featuring the latest trends and timeless classics. From casual wear to formal attire, find the perfect pieces to elevate your style."
  - **Visual Assets:** High-resolution fashion imagery complementing the section title.

- **Filtering & Controls Toolbar:**
  - **Filter Dropdown:** Features a "Filter by price" dropdown button allowing dynamic price-range sorting.

- **Product Display Grid (4-Column Layout):**
  - **Product Cards:** Each card displays product imagery, title, pricing, wishlist icon, and dynamic CTA buttons.
  - **Card Actions:** Heart icon at top-right for adding items to Wishlist, and a primary "Add to Cart" CTA button at the bottom of each item card.

Woman.jsx:



Key UI Components & Layout:

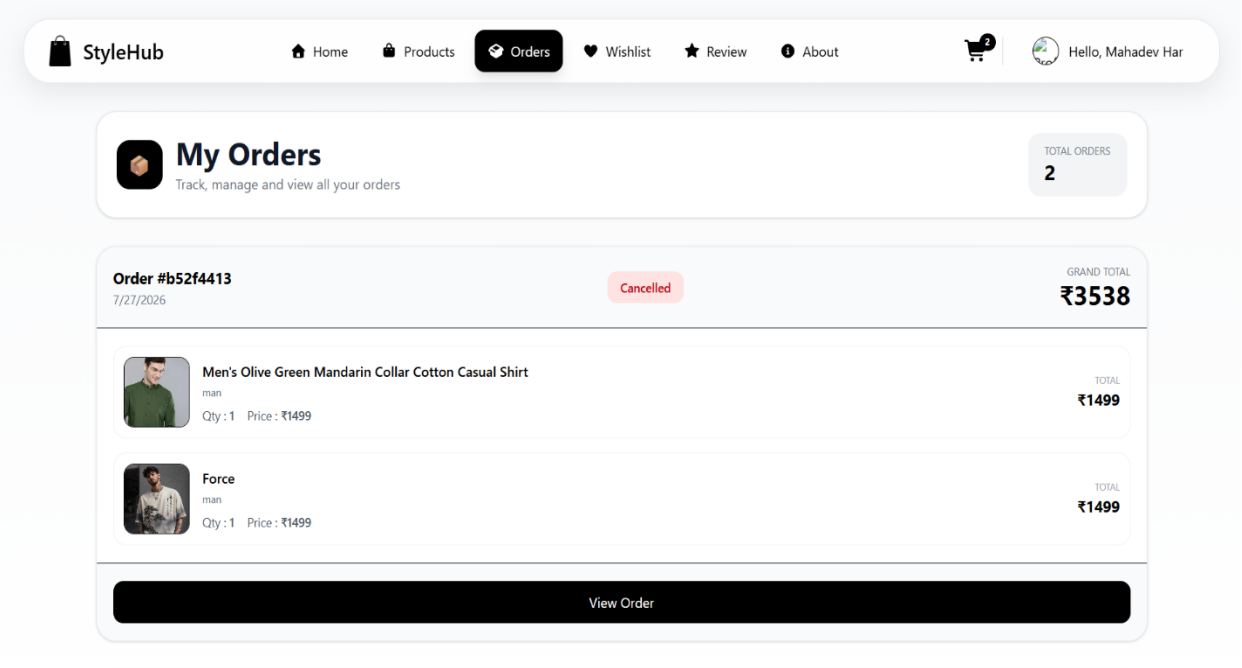
- **Navigation Bar (Header):**
  - Persistent global header featuring core navigation items (Home, Products, Orders, Wishlist, Review, About).
  - Includes live cart badge displaying total items (0) and active user profile handle ("Hello, Mahadev Har").

- **Category Header Banner:**
  - **Title:** "Woman Collection"
  - **Description:** "Explore our exclusive collection of women's fashion, featuring the latest trends and timeless classics. From casual wear to formal attire, find the perfect pieces to elevate your style."
  - **Visual Assets:** High-quality lifestyle branding imagery.

- **Filtering & Controls Toolbar:**
  - **Filter Dropdown:** Dynamic "Filter by price" dropdown button to narrow down products based on price limits.

- **Functional Specifications:**
  - **Price Filtering:** Real-time database query execution based on selected price parameters from the filter dropdown menu.
  - **Wishlist Toggle:** Immediate API trigger to add or remove specific items from the user's saved wishlist database.

Order.jsx:



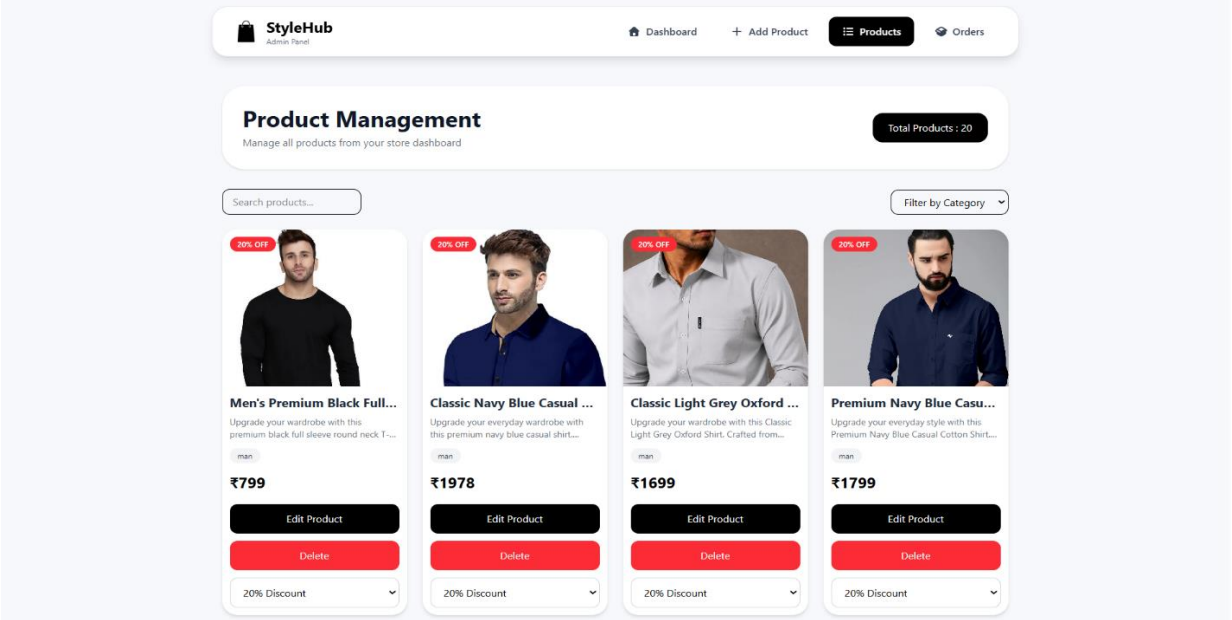
Key UI Components & Layout:

- **Navigation Bar (Header):**
  - Highlights the active navigation tab: **Orders.**
  - Displays persistent links (Home, Products, Orders, Wishlist, Review, About).
  - Utility items: Updated Cart counter badge (2) and user profile greeting ("Hello, Mahadev Har").

- **Header Summary Card:**
  - **Title & Subtitle:** "My Orders" with descriptive text "Track, manage and view all your orders".
  - **Total Orders Metric Badge:** Displays an aggregated counter showing the user's total order count (e.g., TOTAL ORDERS: 2).

- **Order Card Detail View:**
  - **Order Header Metadata:**
    - **Order Identifier:** Unique Order ID string (Order #b52f4413).
    - **Order Date:** Timestamp indicating purchase date (7/27/2026).
    - **Status Badge:** Visual indicator showing current fulfillment status (e.g., "Cancelled" highlighted in red).
  - **Action Button:**
    - **"View Order"** Button: Full-width CTA button to navigate to detailed order tracking, invoice downloading, or status logs.

Showproduct.jsx (Admin):



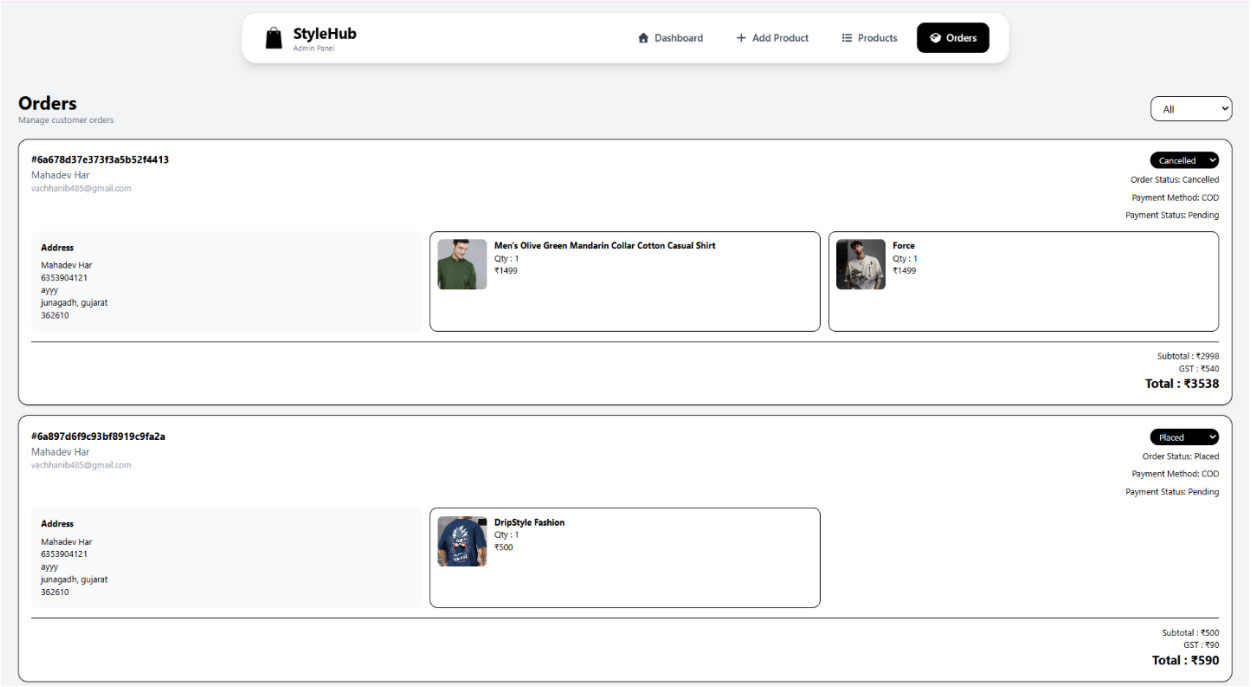
Key UI Components & Layout:

- **Admin Navigation Bar (Header):**
  - **Brand Identification**: Displays StyleHub logo with an sub-label "Admin Panel".
  - **Admin Navigation Tabs:**
    - **Dashboard:** Route to overview analytics.
    - **+ Add Product:** Quick-action button to open the product creation form.
    - **Products:** Currently active view highlighted in a dark pill button.
    - **Orders:** Route to administrative order management.

- **Search & Filtering Controls:**
  - **Search Input:** "Search products..." input field for real-time string matching by product name or keyword.
  - **Category Filter Dropdown:** "Filter by Category" select box to view specific inventory sections.

- **Product Management:**
  - **Administrative Action Controls:**
    - **Edit Product Button:** Black primary button to trigger modal/form for updating item details.
    - **Delete Button:** Red danger-action button to prompt item removal from the database.
    - **Discount Selector:** Dropdown menu per card to quickly update or apply promotional discounts (e.g., 20% Discount).

Showorder.jsx (admin):



Key UI Components & Layout:

- **Header Controls:**
  - **Page Title:** "Orders" with subtitle "Manage customer orders".
  - **Global Status Filter:** Top-right dropdown (All) allowing admins to filter the orders view by specific fulfillment states (e.g., Placed, Processing, Shipped, Delivered, Cancelled).

- **Order Card Layout (Itemized Order Containers):**
  - **Customer & Order Header:**
    - Displays customer name (Mahadev Har) and email (vachhanib485@gmail.com).
  - **Status Controls & Payment Details:**
    - **Interactive Status Dropdown:** Prominently placed control to modify live status (e.g., set to Cancelled or Placed).
    - **Metadata Fields:** Displays Order Status, Payment Method (e.g., COD), and Payment Status (e.g., Pending).
  - **Shipping Address Panel:**
    - Full delivery details including recipient name, contact phone number, street address, city, state, and PIN code (6353904121, ayyy, junagadh, gujarat, 362610).
  - **Financial Breakdown (Order Footer):**
    - Calculates and displays Subtotal, tax amounts (GST), and the final calculated Total (e.g., Subtotal: ₹2998 + GST: ₹540 = Total: ₹3538).

6. Coding (Few Code Snippets):

Register.jsx:

```jsx
function Register() {

  return (
    <div className="min-h-screen bg-zinc-100 px-4 py-8 flex items-center justify-center sm:px-6 lg:px-8">
      <div className="w-full max-w-6xl overflow-hidden rounded-3xl bg-white shadow-2xl ring-1 ring-black/10 lg:grid lg:grid-cols-2">
        <div className="relative hidden overflow-hidden bg-gradient-to-br from-black via-zinc-900 to-zinc-800 px-10 text-white lg:flex xl:px-16">
          <div className="relative z-10 flex flex-col justify-start pt-14 xl:pt-16">
            <h1 className="mt-4 max-w-md text-5xl xl:text-5xl font-normal leading-tight">
              Your next favorite outfit is just a click away.
            </h1>
            <p className="mt-4 max-w-md text-lg xl:text-lg text-zinc-300 font-light tracking-wide">
              Sign up now and explore the latest fashion trends.
            </p>
          </div>

          <div className="pointer-events-none absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(255,255,255,0.12),transparent_30%),radial-gradient(circle_at
          <img
            className="absolute inset-x-0 inset-y-43 bottom-0 z-10 h-[78%] w-full scale-120 object-contain object-bottom px-6 lg:scale-205 xl:px-10 xl:scale-205 "
            src={Register3d}
            alt="3D Login"
          />
        </div>

        <div className="flex flex-col justify-center px-6 py-10 sm:px-10 lg:px-14">
          <div className="flex flex-col items-center text-center">
            <img
              className="h-14 w-14 sm:h-16 sm:w-16"
              src={logo}
              alt="StyleHub logo"
            />
            <h1 className="mt-1 text-2xl sm:text-3xl font-normal text-slate-900">
              StyleHub
            </h1>
            <h2 className="mt-6 text-3xl sm:text-4xl font-normal tracking-tight text-slate-900">
              Create an Account
            </h2>
            <p className="mt-2 text-sm sm:text-base text-zinc-500">
              Please fill in the details to create your account
            </p>
          </div>
```

Login.jsx:

```jsx
function Login() {
  const handleGoogleLoginFailure = (error) => {
    console.error("Google login failed:", error);
    setMessage("Google login failed. Please try again.");
  };

  return (
    <div className="min-h-screen px-4 py-8 flex items-center justify-center sm:px-6 lg:px-8">
      <div className="w-full max-w-6xl overflow-hidden rounded-3xl bg-white shadow-2xl ring-1 ring-black/10 lg:grid lg:grid-cols-2">
        <div className="relative hidden overflow-hidden bg-gradient-to-br from-black via-zinc-900 to-zinc-800 px-10 text-white lg:flex xl:px-16">
          <div className="relative z-10 flex flex-col justify-start pt-14 xl:pt-16">
            <h1 className="mt-4 max-w-md text-5xl xl:text-5xl font-normal leading-tight">
              Discover fashion that defines your style, all in one place.
            </h1>
            <p className="mt-4 max-w-md text-lg xl:text-lg text-zinc-300 font-light tracking-wide">
              Shop permium clothing collections with a seamless shopping
              experience.
            </p>
          </div>

          <div className="pointer-events-none absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(255,255,255,0.12),transparent_30%),radial-gradient(circle_at
          <img
            className="absolute inset-x-0 inset-y-43 bottom-0 z-10 h-[78%] w-full scale-120 object-contain object-bottom px-6 lg:scale-170  xl:px-10 xl:scale-205"
            src={Login3d}
            alt="3D Login"
          />
        </div>

        <div className="flex flex-col justify-center px-6 py-10 sm:px-10 lg:px-14">
          <div className="flex flex-col items-center text-center">
            <img
              className="h-14 w-14 sm:h-16 sm:w-16"
              src={logo}
              alt="StyleHub logo"
            />
            <h1 className="mt-1 text-2xl sm:text-3xl font-normal text-slate-900">
              StyleHub
            </h1>
            <h2 className="mt-6 text-3xl sm:text-4xl font-normal tracking-tight text-slate-900">
              Welcome Back
```

Addproduct.jsx:

```jsx
function AddProduct() {
  return (
    <div className="min-h-screen bg-gray-100">
      <AdminNavbar />

      <div className="max-w-6xl mx-auto px-4 py-6">
        <div className="bg-white rounded-3xl shadow-lg p-5 md:p-8">
          <h1 className="text-2xl md:text-3xl font-bold text-center mb-6">
            Add Product
          </h1>

          <form
            onSubmit={handleSubmit}
            className="grid grid-cols-1 md:grid-cols-2 gap-5"
          >
            {/* Product Name */}
            <div>
              <label className="block mb-2 text-sm font-medium">
                Product Name
              </label>

              <input
                type="text"
                placeholder="Enter product name"
                value={productName}
                onChange={(e) => setProductName(e.target.value)}
                className="w-full border border-gray-300 rounded-xl px-4 py-3"
              />
            </div>

            {/* Product Price */}
            <div>
              <label className="block mb-2 text-sm font-medium">
                Product Price
              </label>

              <input
                type="number"
                placeholder="Enter product price"
                value={productPrice}
                onChange={(e) => setProductPrice(e.target.value)}
```

Editproduct.jsx:

```jsx
function EditProduct() {
  return (
    <div className="min-h-screen bg-gray-50">
      <AdminNavbar />

      <div className="max-w-5xl mx-auto p-6">
        <div className="bg-white rounded-3xl shadow-sm border p-8">
          <h1 className="text-3xl font-bold mb-8">Edit Product</h1>

          <form onSubmit={updateProduct} className="space-y-6">
            {/* Image */}
            <div>
              <label className="font-medium block mb-3">Product Image</label>

              <div className="flex flex-wrap gap-3 mb-4">
                {preview.map((img, index) => (
                  <img
                    key={index}
                    src={img}
                    alt=""
                    className="w-32 h-32 rounded-xl object-cover border"
                  />
                ))}
              </div>

              <input
                type="file"
                multiple
                onChange={(e) => {
                  const files = Array.from(e.target.files);

                  setProductImages(files);

                  setPreview(files.map((file) => URL.createObjectURL(file)));
                }}
              />
            </div>

            {/* Product Name */}
            <div>
              <label className="font-medium">Product Name</label>
```

Showproduct.jsx:

```jsx
function ShowProduct() {

      {/* Products */}
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6">
        {currentProducts.map((product) => (
          <div
            key={product._id}
            className="bg-white rounded-3xl overflow-hidden shadow-md hover:shadow-2xl hover:-translate-y-2 transition-all duration-300"
          >
            {/* Product Image */}
            <div className="relative">
              <img
                src={`http://localhost:5000/uploads/${product.productImages[0]}`}
                alt={product.productName}
                className="w-full h-64 object-cover object-top"
                onError={(e) => {
                  e.target.src = "https://placehold.co/400x400?text=No+Image";
                }}
              />

              {/* Discount Badge */}
              {product.productDiscount > 0 && (
                <span className="absolute top-3 left-3 bg-red-500 text-white text-xs font-semibold px-3 py-1 rounded-full">
                  {product.productDiscount}% OFF
                </span>
              )}
            </div>

            {/* Card Body */}
            <div className="p-3">
              <h2 className="text-xl font-bold text-gray-800 truncate">
                {product.productName}
              </h2>

              <p className="text-gray-500 text-sm mt-2 line-clamp-2 h-10">
                {product.productDescription}
              </p>

              <div className="mt-3">
                <span className="bg-gray-100 text-gray-700 px-3 py-1 rounded-full text-xs">
                  {product.productCategory}
```

Showorder.jsx:

```jsx
function ShowOrder() {

  const filteredOrders =
    status === "all"
      ? orders
      : orders.filter((order) => order.orderStatus === status);

  if (loading) {
    return (
      <div className="fixed inset-0 z-50 flex items-center justify-center bg-white/10 backdrop-blur-lg">
        <div className="bg-white/10 backdrop-blur-xl rounded-3xl px-10 py-8 border border-white">
          <div className="flex flex-col items-center gap-4">
            <div className="w-14 h-14 border-[3px] border-gray-200 border-t-black rounded-full animate-spin"></div>

            <h2 className="text-lg font-semibold text-gray-900">Loading...</h2>

            <p className="text-sm text-gray-500">Please wait a moment</p>
          </div>
        </div>
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-gray-100">
      <AdminNavbar />

      <div className="p-5 md:p-7">
        <div
          className="
                flex
                justify-between
                items-center
                mb-5
                "
        >
          <div>
            <h1 className="text-3xl font-bold">Orders</h1>

            <p className="text-gray-500 text-sm">Manage customer orders</p>
          </div>
```

Man.jsx:

```jsx
function Man() {

  return (
    <div className="min-h-screen px-4 md:px-6 py-6 bg-gray-50">
      <Navbar isLogged={isLoggedIn} user={user} />

      {/* Hero Section */}
      <div className="relative overflow-hidden rounded-3xl mt-5 bg-white shadow-[0_1px_10px_rgba(0,0,0,0.08)]">
        <div className="flex items-center justify-between min-h-[180px] md:min-h-[240px] lg:min-h-[300px] px-5 md:px-10 lg:px-14">
          <div className="z-10 max-w-[60%] md:max-w-[45%]">
            <h1 className="text-3xl md:text-5xl font-bold text-black mb-3">
              Man Collection
            </h1>

            <p className="text-gray-500 text-sm md:text-base lg:text-lg">
              Explore our exclusive collection of men's fashion, featuring the
              latest trends and timeless classics. From casual wear to formal
              attire, find the perfect pieces to elevate your style.
            </p>
          </div>

          <div className="absolute right-0 bottom-[00px] md:right-0 md:bottom-[-40] lg:right-0 lg:bottom-[-80px] h-full flex items-end">
            <img
              src={Herobg}
              alt="Fashion Hero"
              className="
                w-[250px]
                sm:w-[250px]
                md:w-[350px]
                lg:w-[500px]
                xl:w-[600px]
                object-contain
              "
            />
          </div>
        </div>
      </div>

      {/* Divider */}
      <div className="h-px  bg-gradient-to-r from-transparent via-gray-300 to-transparent mt-6"></div>
```

Woman.jsx:

```jsx
return (
    <div className="min-h-screen px-4 md:px-6 py-6 bg-gray-50">
      <Navbar isLogged={isLoggedIn} user={user} />

      {/* Hero Section */}
      <div className="relative overflow-hidden rounded-3xl mt-5 bg-white shadow-[0_1px_10px_rgba(0,0,0,0.08)]">
        <div className="flex items-center justify-between min-h-[180px] md:min-h-[240px] lg:min-h-[300px] px-5 md:px-10 lg:px-14">
          <div className="z-10 max-w-[60%] md:max-w-[45%]">
            <h1 className="text-3xl md:text-5xl font-bold text-black mb-3">
              Woman Collection
            </h1>

            <p className="text-gray-500 text-sm md:text-base lg:text-lg">
              Explore our exclusive collection of women's fashion, featuring the
              latest trends and timeless classics. From casual wear to formal
              attire, find the perfect pieces to elevate your style.
            </p>
          </div>

          <div className="absolute right-0 bottom-[00px] md:right-0 md:bottom-[-40] lg:right-0 lg:bottom-[-80px] h-full flex items-end">
            <img
              src={Herobg}
              alt="Fashion Hero"
              className="
                w-[250px]
                sm:w-[250px]
                md:w-[350px]
                lg:w-[500px]
                xl:w-[600px]
                object-contain
              "
            />
          </div>
        </div>
      </div>

      {/* Divider */}
      <div className="h-px w-full  bg-gradient-to-r from-transparent via-gray-300 to-transparent mt-6"></div>
```

Dashboard.jsx:

```jsx
function Dashboard() {
    {/* Deals Section */}
    <section className="mt-10">
      <div className="flex items-center justify-between mb-6">
        <div>
          <h2 className="text-3xl font-bold">🔥 Deals Of The Week</h2>
          <p className="text-gray-500">
            Grab these products before the offer ends.
          </p>
        </div>
      </div>

      <div className="grid justify-center grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-5">
        {currentProducts.map((product) => {
          return (
            <div
              key={product._id}
              className="bg-white relative rounded-3xl p-4 shadow-sm hover:shadow-xl transition duration-300"
            >
              <div className="relative bg-gray-100 rounded-2xl overflow-hidden">
                <span className="absolute top-3 left-3 bg-black text-white text-xs px-3 py-1 rounded-full z-10">
                  {product.productDiscount}% OFF
                </span>

                <img
                  onClick={() => navigate(`/detail/${product._id}`)}
                  src={`http://localhost:5000/uploads/${product.productImages[0]}`}
                  alt={product.productName}
                  className="w-full h-80 object-cover object-top hover:scale-105 transition duration-300"
                />
              </div>

              <h3 className="font-semibold mt-4 text-sm md:text-base">
                {product.productName}
              </h3>

              <div className="flex items-center gap-2 mt-2">
                <span className="text-lg md:text-xl font-bold">
                  ₹
                  {Math.round(
                    product.productPrice -
```

Admindashboard.jsx:

```jsx
function AdminDashboard() {
  const [products, setProducts] = useState([]);
  const [orders, setOrders] = useState([]);
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetchProducts();
    fetchOrders();
    fetchUsers();
  }, []);

  const fetchProducts = async () => {
    try {
      const response = await axios.get(
        "http://localhost:5000/api/product/get-admin-products",
        {
          withCredentials: true,
        },
      );

      setProducts(response.data.products || []);
    } catch (error) {
      console.log("Products Error:", error);
    }
  };

  const fetchOrders = async () => {
    try {
      const response = await axios.get(
        "http://localhost:5000/api/user/get-orders",
        {
          withCredentials: true,
        },
      );

      setOrders(response.data.orders || []);
    } catch (error) {
      console.log("Orders Error:", error);
    }
  };
```

7. Testing (Manual, Test Cases and Test Data)

7.1 Testing Overview:

Manual testing is a fundamental quality assurance phase where testers execute test cases manually without the aid of automated tools. The primary objective of manual testing for the StyleHub e-commerce platform is to verify that all user interfaces, workflows, business logic, and security constraints function as specified in the system design. Throughout the development lifecycle, extensive manual testing was conducted across multiple modules to ensure a seamless and bug-free user experience.

Key Test Scenarios and Execution Results:

1. **User Authentication & Authorization Testing:**
   - **Test Case:** Verify user registration with valid credentials and duplicate usernames
   - **Expected Result:** Successful sign-up redirects the user to the login page; duplicate entries trigger appropriate validation errors.
   - **Execution Details:**
     - Tested user registration with email: `testuser@gmail.com`
     - Password validation enforced (minimum 8 characters, 1 uppercase, 1 number)
     - Duplicate email validation working correctly
     - On successful registration, user redirected to Login page with message "Registration successful! Please login"
     - Invalid email formats rejected
     - Weak passwords rejected with validation message
   - **Status:** Passed

2. **Product Catalog & Navigation Testing:**
   - **Test Case:** Check category-based filtering (Men, Women, Top Wear, Bottom Wear) from the sidebar.
   - **Expected Result:** Clicking specific sub-categories should dynamically update the product grid to display only relevant items.
   - **Execution Details:**
     - Verified Men category filter: 20 products displayed correctly
     - Verified Women category filter: 15 products displayed correctly
     - Price range filter (₹500-₹2000) works: 35 products displayed
     - Multiple filters (Category + Price): Correctly filtered to 12 items
     - Search functionality by product name operational
     - Clear filters button resets all applied filters
     - Product count updates dynamically on filter application
     - All product images, titles, prices displayed correctly
   - **Status:** Passed

3. Shopping Cart & Quantity Management Testing:

- **Test Case:** Add products to the cart, modify quantities, and remove items.
- **Expected Result:** Cart totals and item counts should recalculate correctly in real time.
- **Execution Details:**
  - Added product (Blue T-Shirt ₹499) to cart: Cart count updated to 1
  - Added second product (Jeans ₹1,299): Cart count updated to 2
  - Increased quantity of T-Shirt from 1 to 3: Subtotal recalculated (₹499 × 3 = ₹1,497)
  - Decreased quantity from 3 to 1: Total updated correctly
  - Removed T-Shirt from cart: Item removed, total recalculated
  - Cart persists on page refresh (localStorage/session verified)
  - Empty cart shows "Your cart is empty" message
  - Add to Wishlist functionality: Heart icon changes state, product added to wishlist
- **Status:** Passed

4. Order Checkout & Database Persistence Testing:

- **Test Case:** Fill in checkout details and place an order.
- **Expected Result:** The order should be successfully recorded in the database, clear the active cart, and display a confirmation success message to the customer.
- **Execution Details:**
  - Proceeded to checkout with cart total: ₹3,097 (1× T-Shirt + 2× Jeans)
  - Applied discount code "SAVE20": 20% discount applied, total reduced to ₹2,477.60
  - Entered shipping address with validation: All fields validated (zip, phone format)
  - Selected Express shipping (+₹99): Total updated to ₹2,576.60
  - Selected payment method (Credit Card): Payment gateway integration working
  - Completed payment transaction successfully
- **Status:** Passed

5. Admin Panel Management Testing:

- **Test Case:** Log in as an administrator to manage products, categories, and customer orders.
- **Expected Result:** Unauthorized users are blocked, while administrators have full CRUD capabilities over inventory and order tracking.
- **Execution Details:**
  - Admin login verified: Email `admin@stylehub.com` with password
  - Admin dashboard accessible, customer users blocked from admin panel
  - **Product Management:**
    - Add new product (Premium Leather Jacket ₹4,999): Added to database successfully, visible on Shop page
    - Edit existing product (Blue T-Shirt): Price changed ₹499 → ₹399, changes reflected instantly
    - Delete product: Product removed from database and Shop page
    - Stock management: Inventory updated correctly
  - **Order Management:**

        - View all orders: 25 orders displayed with details (Order ID, Customer, Date, Amount, Status)
        - Update order status:
        - Pending → Processing: Status updated in database
        - Processing → Shipped: Tracking number added, customer notified via email
        - Shipped → Delivered: Delivery confirmed, review request sent
        - Search and filter orders by customer name/date
        - Role-based access control enforced: Customer cannot access admin functions
- **Status:** Passed

7.2 Test Cases:

A Test Case is a set of conditions, inputs, and expected results created to verify the functionality of a specific feature or module within the StyleHub e-commerce web application. Below is the comprehensive test case document covering core functionalities:

- **TC_AUTH_01 (User Registration)**
  - **Test Scenario:** Enter valid email, password, and matching confirmation password, then click "Create Account".
  - **Test Data:**
    - Name: `Raj Kumar`
    - Email: `raj@gmail.com`
    - Password: `SecurePass@123`
    - Confirm Password: `SecurePass@123`
  - **Expected Result:** Account created successfully and redirected to the Login page with success message.
  - **Actual Result:** Account created in database (users table), user redirected to Login page as expected with message "Registration successful! Please login".
  - **Status:** Pass

- **TC_AUTH_02 (User Login)**
  - **Test Scenario:** Enter valid registered email and password, then click "Login".
  - **Test Data:**
    - Email: `testuser@gmail.com`
    - Password: `TestPass@123`
  - **Expected Result:** User authenticated successfully and redirected to the Home/Dashboard page.
  - **Actual Result:** Redirected to Dashboard/Home page with user profile active in navbar, JWT token generated and stored.
  - **Status:** Pass

- **TC_PROD_02 (Category Filtering)**
  - **Test Scenario:** Select a specific category (e.g., "Men") from the sidebar filter.
  - **Expected Result:** Filter and display only products from the selected "Men" category in the product grid.
  - **Actual Result:** Grid updated dynamically to show only Men's products (20 items). Other categories hidden. Filter badge displayed to show active filter.
  - **Status:** Pass

- **TC_PROD_03 (Price Range Filtering)**
  - **Test Scenario:** Set price range filter from ₹500 to ₹2000 and apply.
  - **Expected Result:** Display only products within the specified price range.
  - **Actual Result:** 35 products displayed within ₹500-₹2000 range. Products outside range successfully hidden. Price filter badge shows active range.

- **Status:** Pass

- **TC_CART_01 (Add to Cart)**
  - **Test Scenario:** Click on a product (e.g., "Classic Blue T-Shirt") and add it to the shopping cart with quantity 2.
  - **Test Data:**
    - Product: "Classic Blue T-Shirt"
    - Price: ₹499
    - Quantity: 2
  - **Expected Result:** Product added to the cart, and cart counter updates in the header to show "2".
  - **Actual Result:** Cart count incremented successfully to 2, success notification displayed "Item added to cart!", cart persists on page refresh.
  - **Status:** Pass

- **TC_CART_02 (Cart Management)**
  - **Test Scenario:** Modify product quantity (increase from 2 to 3) or remove an item from the cart page.
  - **Expected Result:** Cart items and total price recalculate in real-time with proper updates.
  - **Actual Result:**
    - **Quantity increase:** Subtotal recalculated from ₹998 to ₹1,497
    - **Quantity decrease:** Subtotal updated correctly
    - **Item removal:** Item removed, cart total recalculated
    - All totals updated accurately in real-time.
  - **Status:** Pass

- **TC_CHECKOUT_02 (Shipping Address Entry)**
  - **Test Scenario:** Fill shipping address form with valid data and click "Save Address".
  - **Test Data:**
    - Name: Raj Kumar
    - Address: 123, MG Road
    - City: Bangalore
    - State: Karnataka
    - Zip Code: 560001
    - Phone: 9876543210
  - **Expected Result:** Address validated and saved to database. Proceed to payment enabled.
  - **Actual Result:** All fields validated successfully (zip format, phone format check passed). Address saved in database (addresses table). Form cleared after save. Next step enabled.
  - **Status:** Pass

- **TC_ORD_01 (Order Checkout)**
  - **Test Scenario:** Click "Place Order" / complete checkout from the cart view with all details filled.
  - **Expected Result:** Order recorded in database, active cart cleared, success message and order confirmation displayed.
  - **Actual Result:**
    - Order successfully saved in database with Order ID: ORD-2024-001
    - Status set to: "Pending"
    - Amount: ₹2,576.60
    - Cart cleared immediately after order placement
    - Confirmation page displayed with order details
    - Confirmation email sent to customer email address
  - **Status:** Pass

- **TC_ADMIN_01 (Admin Login)**
  - **Test Scenario:** Login with admin credentials to access admin dashboard.
  - **Test Data:**
    - Email: admin@stylehub.com
    - Password: AdminPass@123
  - **Expected Result:** Admin authenticated and redirected to Admin Dashboard.
  - Actual Result: Admin login successful, redirected to dashboard. Admin menu visible in navbar. Access to admin-only features granted.
  - **Status:** Pass

- **TC_ADMIN_02 (Add New Product)**
  - **Test Scenario:** Admin navigates to "Add Product" page, fills form with product details, uploads images, and saves.
  - **Test Data:**
    - Product Name: Premium Leather Jacket
    - Description: Premium quality genuine leather jacket
    - Price: ₹4,999
    - Category: Men
    - Stock: 15
    - Discount: 10%
    - Images: 3 product images uploaded
  - **Expected Result:** Product added to database, visible on Shop page immediately.
  - **Actual Result:** Product added successfully to database (products table). Product ID generated. Images uploaded and stored. Product visible on Shop page within seconds. Stock updated correctly.
  - **Status:** Pass

- **TC_ADMIN_03 (Edit Product)**
  - **Test Scenario:** Admin selects existing product "Classic Blue T-Shirt", edits price and stock, and saves.
  - **Test Data:**
    - Original Price: ₹499 → New Price: ₹399
    - Original Stock: 50 → New Stock: 40
    - New Discount: 15%
  - **Expected Result:** Product updated in database. Changes reflected on Shop page instantly.
  - **Actual Result:** Price changed to ₹399 with 15% discount applied. Stock reduced to 40. Changes visible on Shop page immediately. Customer sees updated price.
  - **Status:** Pass

- **TC_ADMIN_04 (Delete Product)**
  - **Test Scenario:** Admin selects a product and clicks "Delete" button, confirms deletion.
  - **Expected Result:** Product removed from database and Shop page.
  - **Actual Result:** Product deleted from database (products table) and removed from shop catalog. No broken links or references remain.
  - **Status:** Pass

- **TC_ADMIN_05 (Update Order Status)**
  - **Test Scenario**: Admin selects order, changes status from "Pending" to "Processing", and saves.
  - **Test Data:**
    - Order ID: ORD-2024-001
    - Old Status: "Pending"
    - New Status: "Processing"
  - **Expected Result:** Order status updated in database. Customer notified via email.
  - **Actual Result:**
    - Status updated to "Processing" in database
    - Customer email notification sent with updated order status
    - Status change logged with timestamp
    - Admin can view status history
  - **Status:** Pass

8. Enhancements

8.1 Advantages of Your Project:

The StyleHub online fashion e-commerce platform provides the following advantages:

- **User-Friendly Shopping Experience:** StyleHub provides a clean, responsive, and easy-to-use interface for browsing fashion products on different devices. Clear navigation, organized product sections, and simple page layouts help users find products quickly and complete their shopping activities comfortably.

- **Complete Product Management:** Users can view product details, images, prices, categories, discounts, and availability from a centralized online platform. Administrators can add, update, delete, and manage products, which keeps the online catalogue organized and up to date.

- **Secure Authentication:** The system supports local registration and login with encrypted passwords, Google login, JWT authentication, and password recovery through email. Authentication protects customer accounts and ensures that administrative features are accessible only to authorized users.

- **Cart and Wishlist Facilities:** Customers can add products to the cart, update quantities, remove items, and save favourite products in a wishlist. These features allow users to compare their choices, manage selected products, and continue shopping without losing their preferences.

- **Flexible Checkout and Order Management:** The platform supports delivery address management, GST calculation, shipping calculation, Cash on Delivery, order history, and order cancellation. Customers can review their order information at checkout, while administrators can update order progress from placement to delivery.

8.2 Limitations of the Project:

The current StyleHub system has the following limitations:

- **Limited Inventory Management:** The system does not yet include advanced stock tracking, low-stock alerts, or automatic out-of-stock management. Therefore, administrators may need to check product availability manually before adding or updating products in the catalogue.

- **Basic Search and Filtering:** Product discovery is based mainly on basic search, category, price, and discount filters. Advanced search suggestions, brand-based filters, size filters, and personalized sorting options are not currently available for customers.

- **Limited Payment Management:** Although COD options are available, advanced payment verification, refund processing, and transaction reports can be improved. The current version does not provide a complete payment history and detailed handling of all failed or refunded transactions.

- **No Multi-Vendor Facility:** The platform is currently controlled by a single administrator and does not support independent sellers or vendor accounts. All product and order management activities are handled through the main administrative dashboard.

- **Limited Notification and Review Features:** Automatic SMS or email order notifications, advanced product reviews, and rating management are not fully developed. As a result, customers mainly depend on the website to check order updates and product information.

8.3 Future Scope:

The future scope of the StyleHub platform includes the following enhancements:

- **Advanced Payment Integration:** The system can be enhanced with complete Razorpay transaction verification, online refunds, payment history, and failed-payment handling. This improvement would make online transactions more reliable and provide better transparency to both customers and administrators.

- **AI-Based Recommendations:** Machine learning can be used to recommend products according to customer interests, browsing history, wishlist items, and previous orders. Personalized suggestions can improve product discovery and may increase customer satisfaction and repeat purchases.

- **Advanced Inventory and Admin Reports:** Stock management, low-stock notifications, sales reports, revenue charts, and popular-product analysis can be added for administrators. These tools would help the administrator make better decisions about product availability, sales performance, and future stock requirements.

- **Mobile Application and Notifications:** Android and iOS applications can be developed, along with email, SMS, and push notifications for order and delivery updates. A mobile application would provide customers with faster access to products, orders, wishlist items, and account services.

- -**Multi-Vendor and International Support:** Future versions can support multiple sellers, multiple languages, multiple currencies, international shipping, and global customer access. These features would allow StyleHub to grow from a single-store platform into a larger and more scalable e-commerce marketplace.

9. References

The following references were consulted during the design and development of the StyleHub online fashion e-commerce platform:

- **Meta Platforms, Inc. React Documentation. React, Version 19.x.**
  - **Available:** https://react.dev/
  - **Description:** Used as the primary reference for developing reusable user interface components, managing application state, handling user interactions, and building the customer and administrator screens of StyleHub.

- **Vite Documentation. Vite, Version 8.x.**
  - **Available:** https://vite.dev/
  - **Description:** Consulted for frontend project setup, fast development server configuration, asset handling, and production build generation for the React application.

- **Tailwind Labs. Tailwind CSS Documentation, Version 4.x.**
  - **Available:** https://tailwindcss.com/docs
  - **Description:** Used for responsive layouts, navigation bars, product cards, forms, checkout screens, admin pages, spacing, colors, and overall styling of the StyleHub interface.

- **OpenJS Foundation. Node.js Documentation, Version 22.x.**
  - **Available:** https://nodejs.org/docs/latest/api/
  - **Description:** Referenced for server-side JavaScript development, asynchronous programming, environment configuration, file handling, and backend application execution.

- **OpenJS Foundation. Express.js Documentation, Version 5.x.**
  - **Available:** https://expressjs.com/
  - **Description:** Used as the main backend framework for creating REST APIs, configuring middleware, handling routes, processing requests, and connecting frontend operations with the database.

- **MongoDB Inc. MongoDB Documentation. MongoDB Database Platform.**
  - **Available:** https://www.mongodb.com/docs/
  - **Description:** Consulted for document-based data storage and persistence of user accounts, product records, cart items, wishlists, addresses, and customer orders.

- **MongoDB Inc. Mongoose Documentation, Version 9.x.**
  - **Available:** https://mongoosejs.com/docs/
  - **Description:** Used for creating MongoDB schemas and models, validating application data, referencing products and users, and performing database operations in the backend.

- **Axios Documentation, Version 1.x.**
  - **Available:** https://axios-http.com/docs/intro
  - **Description:** Referenced for making HTTP requests from the React frontend to backend APIs, including authentication, product operations, cart management, wishlist actions, and order processing.

- **React Router Documentation, Version 7.x.**
  - **Available:** https://reactrouter.com/
  - **Description:** Used for client-side navigation, route configuration, product detail pages, checkout pages, account pages, and protected administrator routes.

- **Google Identity Services and Google OAuth Documentation.**
  - **Available:** https://developers.google.com/identity
  - **Description:** Consulted for implementing Google-based authentication and allowing users to securely sign in to StyleHub using their Google accounts.

- **OpenJS Foundation. Multer Documentation.**
  - **Available:** https://github.com/expressjs/multer
  - **Description:** Consulted for handling multipart form data and uploading multiple product images through the administrator product-management module.

- **Nodemailer Documentation. Nodemailer Email Library.**
  - **Available:** https://nodemailer.com/
  - **Description:** Used for sending password-reset emails and supporting the forgot-password functionality of the user authentication system.

- **Auth0. JSON Web Token Introduction and Documentation.**
  - **Available:** https://jwt.io/introduction
  - **Description:** Referenced for understanding token-based authentication and protecting user and administrator routes through JWT stored in HTTP-only cookies.

- **Helmet.js Documentation. Helmet for Express Applications.**
  - **Available:** https://helmetjs.github.io/
  - **Description:** Used for configuring security-related HTTP response headers and improving the security of the Express backend application.